

ALMA MATER STUDIORUM · UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica — Scienza e Ingegneria
Corso di Laurea Triennale in Ingegneria e Scienze informatiche

Instradamento Multicast e Soluzioni Parziali in Reti Publish/Subscribe: Un Approccio Basato sul Rilassamento Lagrangiano e su Algoritmo Euristico

Ricerca Operativa: Metodi di Ottimizzazione per le Reti

Relatore:
Chiar.mo Prof.
Marco Antonio Boschetti

Presentata da:
Enrico Ancarani

Sessione Luglio 2026
Anno Accademico 2025/2026

Indice

1	Introduzione	7
1.1	IoT (Internet of Things)	7
1.2	Publish/Subscribe	7
1.3	La Difficoltà del Percorso Ottimo	8
1.3.1	Grafo	8
1.3.2	Centralizzazione	9
1.4	Problema di Ottimizzazione	10
1.4.1	Rilassamento Continuo	10
1.4.2	Rilassamento Lagrangiano	10
1.4.3	Formulazione Matematica del Problema	15
1.5	Ambiente di Sviluppo	15
2	Descrizione del Problema e delle Formulazioni	17
2.1	Scenario del Problema	17
2.2	Simboli Globali	18
2.3	Prima Formulazione	19
2.4	Seconda Formulazione	21
2.5	Rilassamento Lagrangiano della Seconda Formulazione	23
2.5.1	Aggiornamento delle Penalità Lagrangiane	25
3	Miglioramento della Formulazione Rispetto ai Percorsi Parziali	27
3.1	Nuova Formulazione	27
3.2	Mancata Convergenza del Lower Bound alla Soluzione Intera	29
3.2.1	Analisi del Problema nel Dettaglio	29
3.2.2	Scrittura in Formulazione Forte	29
3.2.3	Nuove Condizioni di Attivazione	30
3.2.4	Terza Formulazione Completa	31
4	Implementazione Algoritmica	33
4.1	Implementazione delle Formulazioni Senza Soluzioni Parziali	33
4.1.1	Upper Bound Euristico	33
4.1.2	Prima Formulazione	36
4.1.3	Seconda Formulazione	38
4.2	Formulazione Con Soluzioni Parziali	45
4.2.1	Dijkstra a Scenari e Meccanismo di Discesa dell'Upper Bound	45
4.2.2	Terza formulazione	49
5	Analisi dei Risultati	55
5.1	Scenari e Test Arbitrari	55
5.1.1	Primo Scenario	55

5.1.2	Secondo Scenario	57
5.1.3	Terzo Scenario	59
5.1.4	Quarto Scenario	61
5.1.5	Quinto Scenario	63
5.2	Scenari e Test Casuali	65
5.2.1	Scenario Incrementale Casuale	65
5.2.2	Scenario Incrementale Informazioni e Target	66
5.2.3	Scenario Incrementale Broker e Archi	67
5.3	Analisi della Sensibilità dei Parametri	68
5.3.1	Alpha	68
5.3.2	Qualità dell'Upper Bound	69
5.3.3	Profondità della Ricerca Euristica	70
5.3.4	Conclusioni sui Test	71
6	Conclusioni e Sviluppi Futuri	73

Sommario

IoT (Internet of Things) [1] è l'acronimo utilizzato per indicare la sfera tecnologica moderna che ha alla base l'utilizzo di dispositivi intelligenti per lo scambio di informazioni. All'interno di questa sfera, uno dei metodi di comunicazione più diffusi è quello basato sul modello Publish/Subscribe [2] che consente lo scambio di dati tra dispositivi della rete senza la necessità di conoscenza diretta tra loro. Il modello utilizza tre tipologie di dispositivi: Publisher, Broker e Subscriber. I Publisher, una volta ottenuti dei dati tramite sensori di vario genere, li pubblicano rendendoli disponibili nella rete. I dati si spostano attraverso i Broker, nodi intermedi che gestiscono la connessione, per poi giungere ai Subscriber che li leggono ed elaborano. Il protocollo standard per l'implementazione del modello è l'MQTT [3], progettato per essere leggero e robusto anche su reti instabili.

Questa tesi riprende il lavoro effettuato dal collega Urbinati nell'anno accademico 2021-2022 [4], il quale aveva sviluppato un algoritmo distribuito basato sulle proprietà matematiche del rilassamento lagrangiano [5] dove ogni nodo conosce solamente se stesso e i propri vicini per il calcolo del percorso ottimo. Il presente lavoro reimplementa le formulazioni del collega al fine di analizzare il pensiero logico e i passaggi matematici che hanno portato alla loro costruzione, per poi concentrarsi sull'estensione della seconda formulazione proposta in [4] rendendola capace di gestire scenari di multicasting e di situazioni in cui la consegna completa a tutti i nodi Subscriber non è garantita, casi che risultavano difficili o impossibili da risolvere dal modello originale. Il nuovo modello esteso si fonda su una formulazione forte dei vincoli [6] e introduce una nuova variabile decisionale che consente di rappresentare e gestire la mancata consegna delle informazioni, permettendo di ottenere soluzioni parziali valide anche in presenza di reti con capacità insufficienti o nodi irraggiungibili. Nel nuovo modello viene, inoltre, proposto un meccanismo di penalizzazione che guida l'ottimizzazione verso soluzioni che minimizzano il numero di consegne non effettuate. Per supportare la corretta convergenza del risultato è stato rivisitato l'algoritmo del collega per il calcolo dell'Upper Bound, creandone una nuova versione basata sulla logica di Dijkstra [7] per il calcolo dei cammini minimi, capace di restituire soluzioni parziali.

La tesi è suddivisa in sei capitoli. Il primo capitolo introduce il campo dell'IoT e il modello Publish/Subscribe insieme agli ambienti di sviluppo e alla teoria matematica di riferimento. Il secondo capitolo descrive l'obiettivo della tesi, lo scenario del problema e le formulazioni riprese dalla tesi di Urbinati. Il terzo capitolo tratta lo sviluppo della nuova formulazione estesa, in grado di gestire multicasting e soluzioni parziali. Il quarto capitolo presenta gli algoritmi sviluppati e la logica di funzionamento alla loro base. Il quinto capitolo si pone l'obiettivo di testare gli algoritmi su numerosi casi di test, al fine di comprendere il loro corretto funzionamento e le criticità, con particolare riguardo per la terza formulazione in quanto le altre due sono già state dimostrate funzionanti nella tesi di Urbinati. Il sesto capitolo riassume il lavoro svolto e presenta possibili sviluppi futuri.

In conclusione, il lavoro presentato in questa tesi fornisce due algoritmi per l'ottimizzazione dell'instradamento di informazioni all'interno di reti IoT, superando i limiti delle formulazioni finora proposte e consentendo una gestione flessibile delle informazioni anche in scenari dove un instradamento completo non è sempre possibile.

1 Introduzione

1.1 IoT (Internet of Things)

Il termine IoT [1] nasce come acronimo per indicare quella sfera tecnologica moderna che si basa sull'utilizzo di dispositivi "intelligenti". L'IoT ha avuto particolare rilevanza nell'ultimo periodo storico, andando a rivoluzionare il modo in cui si raccolgono e si processano le informazioni sia in ambito domestico sia industriale.

Un dispositivo definito intelligente o più comunemente chiamato "smart" è un dispositivo in grado di connettersi alla rete per poter svolgere diverse funzioni in cui tra le più importanti è presente lo scambio di informazioni.

In questa tesi ci si sofferma particolarmente sui "sensori", dispositivi chiave per lo scambio di messaggi in una rete IoT. Un sensore è un dispositivo che rileva un cambiamento nel mondo fisico, lo traduce in un segnale digitale e poi lo diffonde ad altri dispositivi, i quali lo utilizzeranno per prendere decisioni.

Una possibile criticità risiede proprio in questa parte del funzionamento, se una decisione deve essere presa con tempestività, anche la rete di trasmissione dovrà essere veloce e efficiente.

1.2 Publish/Subscribe

Tra le metodologie più famose di trasmissione di messaggi, quella che si intende approfondire è il Publish/Subscribe [2], un paradigma particolarmente adatto all'IoT, nato come sostituzione del classico Client-Server. In questo schema i dispositivi si dividono in Publisher e Subscriber collegati tra loro tramite dei nodi/server di trasmissione definiti Broker.

In questo modello ognuno ha il suo ruolo: i Publisher devono diffondere nella rete le informazioni in loro possesso, i Broker devono ritrasmettere queste informazioni senza permettere danneggiamenti o perdita delle stesse e i Subscriber sono i ricevitori o i richiedenti. Per quanto riguarda il meccanismo alla base è molto semplice: il Publisher invia messaggi etichettati con un topic, il Broker li riceve e li inoltra solamente ai Subscriber iscritti a quel topic.

Questo approccio permette un disaccoppiamento tra nodo di partenza e di arrivo, in quanto le due parti non si conoscono direttamente e non devono per forza essere attive contemporaneamente per garantire il successo della comunicazione.

Il protocollo standard per l'implementazione di questo modello è l'MQTT [3], progettato per essere leggero e robusto anche su reti instabili ma che presenta alla base una criticità importante: essendo il Broker il nodo centrale di comunicazione, la sua irraggiungibilità diventa rischio di interruzione per l'intera rete [8]. Per ovviare al problema, si è optato per la creazione di infrastrutture con più Broker configurati per lavorare e apparire come uno solo, in modo che nel caso di fallimento di un server, i rimanenti possano continuare il lavoro.

Facendo riferimento al paragrafo precedente si può associare il ruolo di Publisher

ai sensori e il ruolo di Subscriber a tutti quei dispositivi che necessitano del valore rilevato.

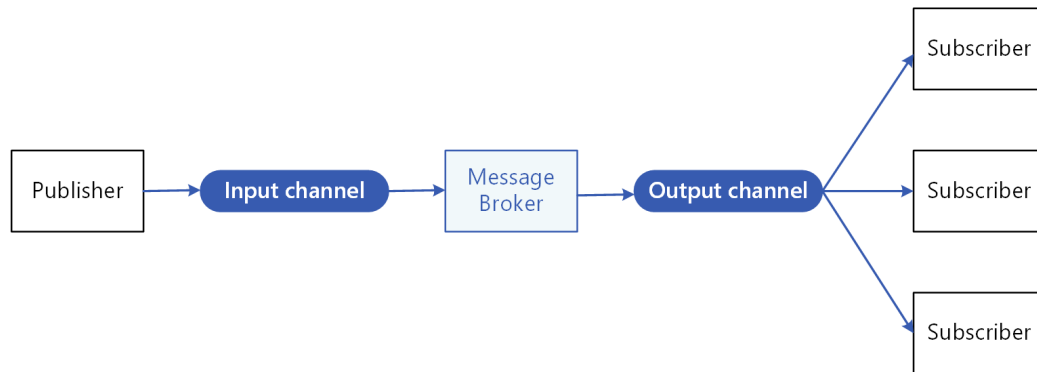


Figura 1.1: Immagine di funzionamento di una rete Publish/Subscribe

1.3 La Difficoltà del Percorso Ottimo

In un sistema moderno, per garantire la velocità di una rete è cruciale trovare il percorso ottimo per la trasmissione dei dati, infatti una scelta non ottimizzata potrebbe ritardare o addirittura impedire la ricezione delle informazioni, compromettendo il funzionamento dei dispositivi che dipendono da esse. Per questo motivo è necessario individuare il metodo più efficiente per rappresentare la rete e per calcolare il percorso ottimale.

1.3.1 Grafo

Uno dei metodi migliori per poter descrivere una rete di dispositivi connessi è tramite un grafo. Usando questa struttura, i dispositivi vengono rappresentati tramite dei nodi e i collegamenti tramite degli archi.

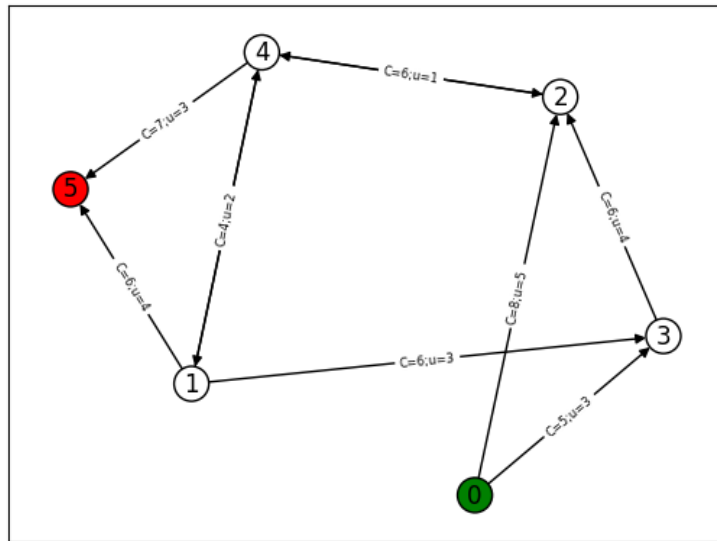


Figura 1.2: Immagine di un grafo con costi e capacità degli archi

L'uso di un grafo risulta una scelta efficiente perché, per reti semplici, consente di mostrare in modo naturale sia i componenti della rete sia le relazioni tra essi e grazie alla sua flessibilità permette inoltre di aggiungere varie proprietà agli elementi rappresentati, rendendo possibile modellare scenari complessi con facilità.

1.3.2 Centralizzazione

Un altro aspetto fondamentale del problema riguarda la centralizzazione dell'elaborazione della rete. In un caso reale, calcolare qual è il percorso ottimo necessita di una notevole potenza di calcolo e, soprattutto, bisogna che ci sia la garanzia di possedere informazioni complete sulla composizione della rete. Da qui nasce l'idea di non affidarsi più esclusivamente ad un server centrale ma di distribuire il calcolo su tutti i dispositivi della rete rendendola autonoma sia nella ricerca del percorso ottimo sia nella gestione dei guasti in caso di malfunzionamento di un canale.

Peer-To-Peer

Ciò è reso possibile sfruttando un protocollo Peer-to-Peer [9], ovvero un protocollo che tratta i dispositivi in comunicazione sia come Client sia come Server, permettendogli dunque di scambiarsi richieste di risorse o servizi senza dover passare da terzi.

1.4 Problema di Ottimizzazione

Il problema della ricerca del percorso ottimo può essere formalizzato come un problema di ottimizzazione [6], problema che richiede di trovare la migliore soluzione possibile fra tutte le soluzioni ammissibili, massimizzando o minimizzando una funzione definita obiettivo e rispettando i vincoli imposti.

Questo tipo di problema è descrivibile utilizzando un modello matematico:

$$\min/\max \sum_i c_i x_i \quad (1.1)$$

vincoli:

$$Ax \geq b \quad (1.2)$$

$$Bx \geq d \quad (1.3)$$

$$x \in \{0, 1\} \quad (1.4)$$

Con c il vettore dei costi, x la variabile decisionale, A e B le matrici dei coefficienti del problema, b e d i vettori delle costanti per i vincoli.

Molto spesso quando si tratta di problemi di questo tipo, si possono riscontrare delle difficoltà nella risoluzione, per questo viene adottata la tecnica del rilassamento dei vincoli che consente di semplificare il calcolo ottenendo un limite inferiore per la soluzione del problema originale.

1.4.1 Rilassamento Continuo

Il rilassamento più intuitivo è quello continuo [6], dove se il problema presenta la variabile decisionale come intera, è possibile costruire un problema di ottimizzazione equivalente, abbandonando il vincolo di interezza della variabile e permettendole di assumere un qualsiasi valore reale all'interno di un intervallo continuo derivato dal vincolo originale.

Questo approccio trasforma il problema intero in un problema di programmazione lineare continua, per il quale esistono algoritmi efficienti. Lo spazio delle soluzioni ammissibili, tuttavia, è più ampio e il valore ottimo del problema rilassato potrebbe non essere sempre applicabile al problema originale.

$$x \in \{0, 1\} \rightarrow 0 \leq x \leq 1 \quad (1.5)$$

1.4.2 Rilassamento Lagrangiano

Un rilassamento particolarmente utile è quello lagrangiano [5], che permette di rilassare i vincoli computazionalmente dispendiosi aggiungendoli alla funzione

obiettivo e associando a ognuno di essi un moltiplicatore di Lagrange, definito anche penalità lagrangiana. Questi moltiplicatori si adattano al rispetto del vincolo rilassato: maggiore è la violazione, maggiore sarà la distanza del valore della funzione obiettivo dall'ottimo. Il risultato è, dunque, una nuova funzione obiettivo, detta lagrangiana, che dipende sia dalle variabili originali del problema sia dai moltiplicatori di Lagrange, e la cui risoluzione fornirà i valori ottimi di entrambi.

Spiegazione Matematica

Si considera un problema di minimizzazione P:

$$z(P) = \min cx \quad (1.6)$$

vincoli:

$$Ax \geq b \quad (1.7)$$

$$x \in \{0, 1\} \quad (1.8)$$

Con c il vettore dei costi, x la variabile decisionale, A la matrice dei coefficienti del problema, b il vettore delle costanti per il vincolo.

Con il rilassamento lagrangiano si rilassa il vincolo $Ax \geq b$ aggiungendo una penalità λ non negativa e inserendolo all'interno della funzione obiettivo.

$$z(P) = \min cx, Ax - b \geq 0 \quad (1.9)$$

↓

$$L(\lambda) = \min cx - \lambda(Ax - b) \quad (1.10)$$

vincoli:

$$x \in \{0, 1\} \quad (1.11)$$

Il motivo del segno $-$ è dovuto al fatto che il vincolo è soddisfatto quando assume un valore maggiore o uguale a 0, quindi dato che λ è definita non negativa, se il vincolo non viene soddisfatto come nel caso in cui sia negativo, sottrarlo dalla funzione obiettivo ne aumenta il valore, allontanandolo dal minimo ricercato.

Il valore ottimale del problema originale $z(P)$ sarà maggiore o al massimo uguale al valore ottimale del problema rilassato $L(\lambda)$.

$$z(P) \geq L(\lambda), \quad \forall \lambda \geq 0 \quad (1.12)$$

Questo perché non avendo l'obbligo di rispettare il vincolo, che è stato rilassato, lo spazio delle soluzioni ammissibili è più ampio e quindi si possono trovare soluzioni che il problema originale non può considerare corrette, le quali, avendo meno costrizioni, avranno un valore più basso o al massimo uguale alla soluzione dell'originale.

Dimostrazione:

1. Si prende una x^* che è soluzione ottima di $z(P)$ quindi è ammissibile e il vincolo è rispettato.
2. Per $\lambda \geq 0$ e $Ax^* - b \geq 0$ si ha:

$$cx^* \geq cx^* - \lambda(ax^* - b) \quad (1.13)$$

3. x^* è anche soluzione ammissibile per il problema rilassato quindi esiste sicuramente una soluzione di $L(\lambda)$ che sarà o minore o uguale al valore di x^* :

$$L(\lambda) = \min(cx - \lambda(Ax - b)) \leq cx^* - \lambda(Ax^* - b) \quad (1.14)$$

4. Unendo le diseguaglianze:

$$z(P) = cx^* \geq cx^* - \lambda(ax^* - b) \geq L(\lambda) \rightarrow z(P) \geq L(\lambda) \quad (1.15)$$

Grazie a questa proprietà, si può pensare di massimizzare la funzione lagrangiana per ottenere il migliore limite inferiore possibile per il problema originale e si può descrivere con la formula del duale lagrangiano:

$$z(D_L) = \max_{\lambda \geq 0} [L(\lambda)] \quad (1.16)$$

Una delle tecniche che può essere usata per la ricerca della soluzione del duale lagrangiano è il metodo del subgradiente.

Metodo del Subgradiente

Il metodo del subgradiente [5] è un metodo iterativo usato per massimizzare una funzione concava, come quella lagrangiana, con lo scopo di ottenere il migliore limite inferiore per il problema originale. Nello specifico, si può utilizzare per calcolare i moltiplicatori di Lagrange associati ai vincoli rilassati che forniscono il risultato $L(\lambda)$ più alto.

La funzione lagrangiana non è perfettamente una curva liscia ma è composta di segmenti, ciascuno corrispondente a una soluzione ottima $\bar{x}$ per il λ specificato.

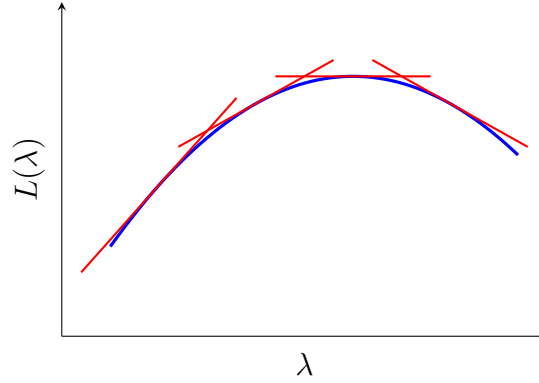


Figura 1.3: Immagine di una funzione lagrangiana con involucro

Data una λ , si ottiene una $\bar{x}$ tale che:

$$L(\bar{\lambda}) = \min(cx - \bar{\lambda}(Ax - b)) = c\bar{x} - \bar{\lambda}(A\bar{x} - b) \quad (1.17)$$

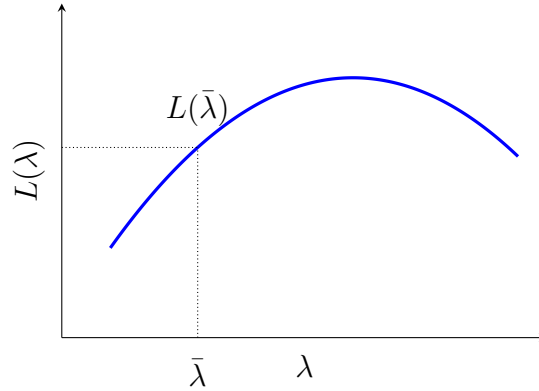


Figura 1.4: Valore di $L(\bar{\lambda})$ corrispondente a una $\bar{\lambda}$ scelta

Riprendendo la soluzione $\bar{x}$ derivata dalla $\bar{\lambda}$ data, si può ridefinire $L(\lambda)$, cioè il valore ottimo in una qualsiasi altra λ generica, come o minore o uguale della soluzione trovata ora:

$$L(\lambda) = \min(cx - \lambda(Ax - b)) \leq c\bar{x} - \lambda(A\bar{x} - b) \quad (1.18)$$

Questo perché la disequazione confronta elementi diversi, nel lato sinistro è presente una x ottima per la nuova λ , mentre nel lato destro è presente la $\bar{x}$, ottima per $\bar{\lambda}$ ma non necessariamente per λ . Poiché le si stanno valutando con la stessa λ , allora per definizione, la x ottima darà un valore minore o, nel caso in cui anche $\bar{x}$

sia ottima per λ , uguale.

Sottraendo ora l'equazione (1.17) dalla (1.18) si ottiene:

$$L(\lambda) - L(\bar{\lambda}) \leq c\bar{x} - \lambda(A\bar{x} - b) - (c\bar{x} - \bar{\lambda}(A\bar{x} - b)) \quad (1.19)$$

$\downarrow$

$$L(\lambda) - L(\bar{\lambda}) \leq -\lambda(A\bar{x} - b) + \bar{\lambda}(A\bar{x} - b) \quad (1.20)$$

$\downarrow$

$$L(\lambda) - L(\bar{\lambda}) \leq (A\bar{x} - b)(-\lambda + \bar{\lambda}) \quad (1.21)$$

$\downarrow$

$$L(\lambda) \leq L(\bar{\lambda}) + (A\bar{x} - b)(-\lambda + \bar{\lambda}) \quad (1.22)$$

Si definisce $s = -(A\bar{x} - b)$ e si trasforma l'equazione:

$$L(\lambda) \leq L(\bar{\lambda}) + s(\lambda - \bar{\lambda}) \quad (1.23)$$

L'obiettivo della risoluzione del duale lagrangiano è, a ogni iterazione, di far crescere $L(\lambda)$ fino a trovare il valore massimo. In termini matematici si vuole che $L(\lambda)$ sia maggiore di $L(\bar{\lambda})$ e che quindi la loro differenza sia positiva. L'unico modo di ottenere questo risultato è di avere il termine: $s(\lambda - \bar{\lambda}) > 0$ e che quindi sommandolo a $L(\bar{\lambda})$ ne aumenti il valore. Per garantire che ciò accada si opta per spostare λ rispetto a $\bar{\lambda}$ usando lo stesso verso del subgradiente s in modo che:

$$\lambda = \bar{\lambda} + \theta s \quad (1.24)$$

$\downarrow$

$$\lambda - \bar{\lambda} = \theta s \quad (1.25)$$

$$s(\lambda - \bar{\lambda}) = s(\theta s) = \theta s^2 \quad (1.26)$$

Di conseguenza per un $s \neq 0$ e per un passo $\theta > 0$ il valore risultante sarà sicuramente positivo e porterà ad una λ maggiore rispetto a $\bar{\lambda}$.

Anche la scelta del parametro θ è fondamentale, un passo troppo grande potrebbe far saltare la soluzione ottima portando alla divergenza, mentre un passo troppo corto potrebbe convergere alla soluzione ottima troppo lentamente.

1.4.3 Formulazione Matematica del Problema

Nei problemi di ottimizzazione, uno degli aspetti più cruciali risiede nel modo in cui le relazioni tra le variabili vengono tradotte in vincoli. Uno stesso problema può essere modellato attraverso diversi insiemi di vincoli, ciascuno dei quali si definisce come una formulazione del problema [6].

Due formulazioni si definiscono equivalenti se possiedono lo stesso identico insieme di soluzioni ammissibili intere. Tuttavia, nonostante l'equivalenza sull'insieme delle soluzioni, le loro prestazioni computazionali possono differire di molto, rendendo la scelta della formulazione un fattore chiave per la qualità del modello matematico creato.

Debole Contro Forte

I risolutori, nel cercare la soluzione ottima, non operano direttamente con le variabili intere, ma applicano il rilassamento continuo permettendo loro di assumere valori frazionari. È proprio durante questa fase di calcolo che si distinguono due tipologie di formulazioni diverse:

- Formulazione debole [6]: Dopo aver effettuato il rilassamento, il risolutore è libero di assegnare alle variabili valori frazionari molto distanti dai limiti interi reali. Di conseguenza il modello può ottenere un valore ottimo rilassato anche molto distante dall'ottimo intero.
- Formulazione forte [6]: I vincoli sono scritti in modo tale da limitare pesantemente le scelte frazionarie favorendo l'ottenimento di valori interi per le variabili e di conseguenza, un valore ottimo molto vicino all'ottimo intero se non coincidente.

1.5 Ambiente di Sviluppo

Esistono molti ambienti di sviluppo che permettono la risoluzione di problemi di ottimizzazione, ma quelli su cui ci si vuole soffermare in questa tesi sono Cplex Optimization Studio e Google Colab.

Cplex Optimization Studio [10] è un software sviluppato da IBM con l'obiettivo di permettere la risoluzione di problemi di ottimizzazione in maniera intuitiva senza l'obbligo di dover implementare i metodi risolutivi da parte dell'utente finale. Per garantire semplicità è richiesta solamente la struttura del problema, i dati e la configurazione di avvio. È un software potente ma presenta delle limitazioni: è particolarmente costoso in termini economici, usa un linguaggio di programmazione molto specifico, non facilmente riconoscibile a prima vista da un utente

inesperto e modellare problemi di rilassamento lagrangiano [5] al suo interno è complesso.

Google Colab [11] invece è un ambiente Python online scelto per diversi vantaggi: le librerie sono già installate o facilmente installabili, i file sono salvati nel cloud, quindi accessibili da più dispositivi e la modalità a celle permette di eseguire porzioni di codice senza dover rieseguire tutto ogni volta. La scelta di Python è dovuta sia a delle difficoltà tecniche nel far collaborare l'ambiente di sviluppo Cplex con Java, sia al fatto che si parla di un ambiente estremamente flessibile e in quanto tale, permette di importare numerose librerie per la semplificazione dei calcoli. È grazie a questa flessibilità che l'ambiente di Google Colab permette di modellare anche problemi di ottimizzazione complessi, come il rilassamento lagrangiano, e permette la visualizzazione delle casistiche tramite librerie grafiche come Networkx [12] e Matplotlib.pyplot [13].

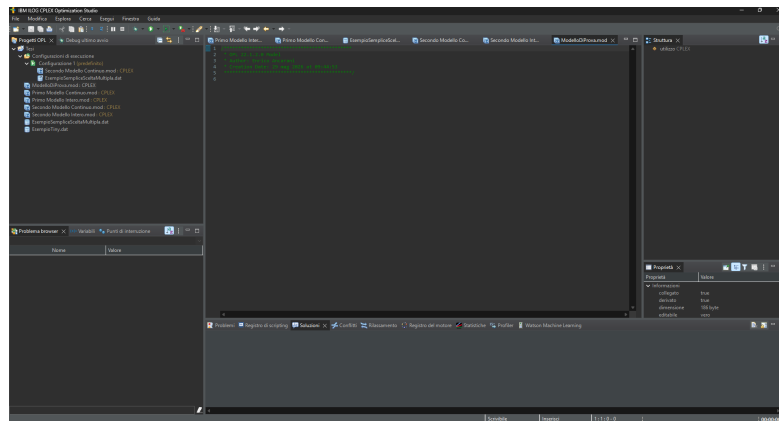


Figura 1.5: Immagine dell'ambiente di sviluppo di Cplex Optimization Studio

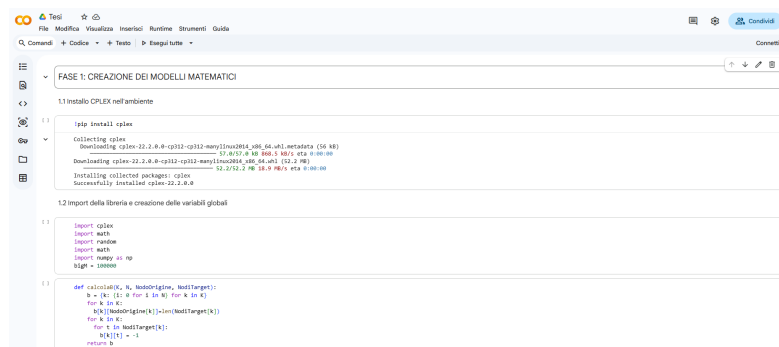


Figura 1.6: Immagine dell'ambiente di sviluppo di Google Colab

2 Descrizione del Problema e delle Formulazioni

2.1 Scenario del Problema

Il problema che si vuole approfondire nello specifico è quello di una rete IoT [1] in cui si vogliono scambiare informazioni tra generatori di dati e ricevitori. La rete è caratterizzata da una struttura Publish/Subscribe [2] che, come definita in precedenza, presenta al suo interno dei nodi di comunicazione definiti Broker, sensori che svolgono il ruolo di generatori dei dati (Publisher) e utenti finali che svolgono il ruolo di ricevitori (Subscriber).

L'obiettivo della tesi è sviluppare, a partire dal lavoro del collega Urbinati [4] svolto nell'anno accademico 2021-2022, un algoritmo centralizzato efficiente per il calcolo del costo del percorso ottimo. La tesi del collega prevedeva di creare due formulazioni del medesimo problema, sviluppando in particolar modo la seconda in maniera distribuita sfruttando un protocollo Peer-to-Peer [9]. L'idea è di riprendere le formulazioni create, analizzarne i passaggi logici e matematici che hanno portato alla loro costruzione e poi di estendere la seconda per poter gestire sia scenari di multicasting, ovvero situazioni in cui la stessa informazione è richiesta contemporaneamente da più utenti, sia il calcolo di percorsi parziali in presenza di destinatari non raggiungibili, casistiche che il modello originale non supportava.

Per modellare correttamente il problema è necessario considerare un caso reale, quindi con diversi fattori in gioco:

- Capacità del canale (quante informazioni possono passare insieme nello stesso collegamento nello stesso momento).
- Costo del canale (quanto è dispendioso in termini economici o computazionali utilizzare quel determinato percorso).
- Peso di un'informazione (quanta banda occupa la singola informazione all'interno del canale).

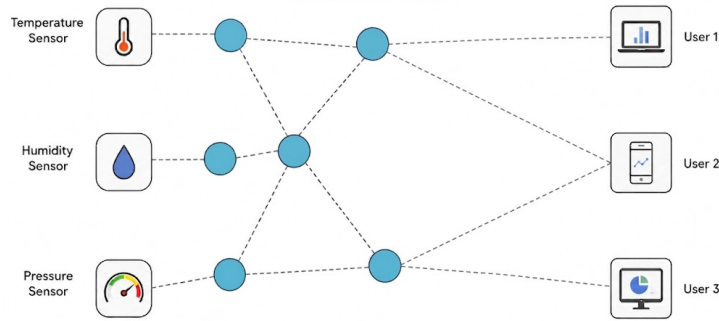


Figura 2.1: Immagine di una rete con dei Publisher (sensori), dei Broker e dei Subscriber (utenti finali)

I seguenti simboli globali e le successive due formulazioni sono ripresi dalla tesi del collega, con una particolare attenzione nella spiegazione di tutti i passaggi, anche quelli da lui omessi.

2.2 Simboli Globali

Per poter iniziare una transizione del problema dal mondo reale a una formulazione matematica è necessario mappare le variabili reali in un contesto informatico/matematico:

- K , è l'insieme contenente le informazioni da inviare o ricevere.
- N , è l'insieme contenente tutti i nodi della rete.
- *Publisher*, è l'insieme contenente i nodi che secondo il modello Publish/Subscribe fanno parte dei generatori delle informazioni.
- *Broker*, è l'insieme contenente i nodi di mezzo che non devono né generare né consumare le informazioni ma solo ritrasmettere.
- *Subscriber*, è l'insieme contenente i nodi che secondo il modello Publish/Subscribe devono ricevere le informazioni.
- A , è l'insieme contenente gli archi della rete quindi i canali di comunicazione.
- $NodoOrigine_k$, è l'insieme contenente per ogni informazione k il suo nodo di origine.

- $NodiTarget_k$, è l'insieme contenente per ogni informazione k i nodi che richiedono quella informazione.
- b_i^k , è una variabile che, a seconda del nodo i e della informazione k , indica il ruolo di i rispetto a k : se è maggiore di 0 allora i è un generatore e produce b_i^k informazioni k , se è uguale a 0 allora i è un nodo di trasmissione e invece se è uguale a -1 allora i è un nodo richiedente di k .
- a_k , è una variabile che indica per ogni informazione k quanta banda occupa all'interno di un canale.
- c_{ij} , è l'insieme contenente per ogni arco i, j il suo costo.
- u_{ij} , è l'insieme contenente per ogni arco i, j la sua capacità.
- Lt_i , è l'insieme contenente per ogni nodo i tutti i nodi raggiungibili da i .
- Lr_i , è l'insieme contenente per ogni nodo i tutti i nodi da cui si può raggiungere i .
- x_{ij}^k , è una variabile decisionale che indica quanti Subscriber, richiedenti l'informazione k , stanno venendo serviti usando l'arco i, j .
- e_{ij}^k , è una variabile decisionale che indica se per il trasporto di una informazione k l'arco i, j è attivo.
- M , è una variabile di valore molto alto, solitamente pari al numero dei nodi, usata per definire dei vincoli.

2.3 Prima Formulazione

Una prima formulazione può essere descritta con uno schema minimum-cost maximum-flow [14], un problema comunemente noto nell'ambito dell'ottimizzazione dove viene richiesto di soddisfare il maggior numero di richieste possibili minimizzando i costi nel farlo.

Funzione Obiettivo:

$$\text{minimizza } \sum_{k \in K} \sum_{i \in N} \sum_{j \in Lr_i} c_{ji} x_{ji}^k \quad (2.1)$$

Vincoli:

$$\sum_{j \in Lr_i} x_{jt}^k = 1 \quad \forall t \in NodiTarget_k, k \in K \quad (2.2)$$

$$\sum_{j \in Lr_i} x_{ji}^k = \sum_{j \in Lt_i} x_{ij}^k \quad \forall i \in Broker, k \in K \quad (2.3)$$

$$Me_{ij}^k \geq x_{ij}^k \quad \forall (i, j) \in A, k \in K \quad (2.4)$$

$$\sum_{k \in K} a^k (e_{ij}^k + e_{ji}^k) \leq u_{ij} \quad \forall i \in N, j \in Lt_i \quad (2.5)$$

$$x_{ij}^k \geq 0 \quad \forall (i, j) \in A, k \in K \quad (2.6)$$

$$e_{ij}^k \in \{0, 1\} \text{ oppure } 0 \leq e_{ij}^k \leq 1 \quad \forall (i, j) \in A, k \in K \quad (2.7)$$

Alla base di questa formulazione si immagina che ogni copia dell'informazione abbia un costo e che, quindi, per ogni Subscriber soddisfatto aumenti la spesa totale, ma poiché la copia del messaggio viene generata solo dopo il suo transito, la capacità prende in considerazione solo se l'arco è attivo o meno. Questa formulazione nasce con l'idea di un approccio centralizzato e richiede perciò un computer centrale per il calcolo dell'ottimo.

(2.1): Per ogni informazione k , per ogni nodo i , si somma il costo degli archi entranti in i moltiplicati per il numero di Subscriber serviti su quell'arco. In pratica il costo totale dipende da quanti Subscriber vengono serviti per ogni informazione k e da quali archi si sta utilizzando per farlo.

(2.2): Sommando le informazioni in entrata al Subscriber il risultato deve corrispondere a 1. Questo vincolo permette di garantire che ogni Subscriber riceva l'informazione che ha richiesto, né di meno né di più. Questo deve valere per ogni Subscriber e per ogni informazione richiesta.

(2.3): Sommando il numero di Subscriber serviti tramite gli archi entranti nel Broker i , il risultato deve essere uguale al numero di Subscriber serviti tramite gli archi uscenti dal medesimo nodo. Questo vincolo garantisce che i sia esclusivamente un nodo di passaggio e che non possa generare o consumare informazioni.

(2.4): Se si sta soddisfacendo un Subscriber tramite l'arco i, j allora quest'ultimo deve essere considerato attivo rispetto al transito dell'informazione k . Questo vincolo serve a legare la variabile e con la variabile x , garantendo che se si sta usando

un arco allora esso è attivo. La scelta di M ricade sul numero totale dei nodi poiché x , potendo assumere al massimo un valore pari al numero di nodi Subscriber, non potrà mai superare tale soglia.

(2.5): Per ogni arco i, j si controlla che la somma della banda occupata da tutte le informazioni passanti in qualsiasi direzione non sia mai maggiore della capacità totale. Questo vincolo garantisce di non superare mai il valore massimo della capacità di un arco.

(2.6): Per ogni arco i, j e per ogni informazione k , la variabile x deve essere non negativa in quanto rappresenta il numero di Subscriber serviti tramite quell'arco e per quella informazione.

(2.7): Per ogni arco i, j e per ogni informazione k , la variabile e indica se l'arco è attivo per il transito dell'informazione. Nel caso di modello intero assume valore binario mentre nel caso di rilassamento continuo può assumere qualsiasi valore reale nell'intervallo $[0, 1]$.

2.4 Seconda Formulazione

Una seconda formulazione del problema si può basare sull'idea che la generazione dell'informazione sia priva di costo e, quindi, che la spesa totale sia associata esclusivamente all'attivazione degli archi. Un obiettivo raggiungibile con questa formulazione è quello di essere facilmente rilassabile con il metodo lagrangiano, permettendo in questo modo di creare sotto-problemi locali, ovvero problemi dipendenti esclusivamente dal nodo su cui sono definiti insieme agli archi e ai nodi adiacenti.

Funzione Obiettivo:

$$\text{minimizza } \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k \quad (2.8)$$

Vincoli:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k = b_i^k \quad \forall i \in N - \{NodoOrigine_k\}, k \in K \quad (2.9)$$

$$\sum_{k \in K} a^k e_{ij}^k \leq u_{ij} \quad \forall (i, j) \in A \quad (2.10)$$

$$e_{ij}^k \leq x_{ij}^k \leq e_{ij}^k |NodiTarget_k| \quad \forall (i, j) \in A, k \in K \quad (2.11)$$

$$x_{ij}^k \geq 0 \quad \forall (i, j) \in A, k \in K \quad (2.12)$$

$$e_{ij}^k \in \{0, 1\} \text{ oppure } 0 \leq e_{ij}^k \leq 1 \quad \forall (i, j) \in A, k \in K \quad (2.13)$$

Questa formulazione nasce con l'idea di un possibile approccio decentralizzato e rende possibile perciò effettuare il calcolo dell'ottimo all'interno dei nodi distribuiti.

(2.8): Per ogni informazione k , si somma il costo dell'arco per la sua attivazione rispetto al trasporto di k . Minimizzando si cerca di usare il minor numero di archi possibili soddisfacendo comunque tutte le richieste.

(2.9): Per ogni nodo i e per ogni informazione k , la differenza tra le informazioni uscenti e entranti deve essere uguale a b . Questo vincolo garantisce che ogni nodo svolga correttamente il proprio ruolo all'interno della rete per ogni informazione considerata. La creazione di questo vincolo nasce con l'intenzione di unire le condizioni (2.2) e (2.3) della prima formulazione in modo da avere un solo vincolo che analizza sia gli archi entranti sia quelli uscenti, semplificando il rilassamento lagrangiano.

(2.10): Per ogni arco i, j si verifica che la somma della banda occupata dalle informazioni passanti non sia mai maggiore della capacità massima. La differenza del vincolo rispetto alla versione della prima formulazione (2.5) risiede nella gestione delle capacità: anziché considerare ogni arco come singolo e bidirezionale con una capacità condivisa tra le due direzioni, ora si considera come due archi distinti monodirezionali con capacità indipendenti. Questa scelta viene effettuata per semplificare il rilassamento lagrangiano, rendendo il vincolo dipendente dal singolo arco.

(2.11): Per ogni arco i, j e per ogni informazione k , il valore della variabile x è legato alla variabile e : se l'arco è disattivato nessuna informazione può transitarvi, altrimenti se è attivo il numero di informazioni può essere scelto in un intervallo da uno fino al numero di Subscriber richiedenti. Questo vincolo garantisce un legame

tra l'attivazione di un arco e il numero di informazioni che passano al suo interno. I vincoli (2.12) e (2.13) sono analoghi ai vincoli (2.6) e (2.7).

2.5 Rilassamento Lagrangiano della Seconda Formulazione

La seconda formulazione, con le modifiche apportate ai vincoli rispetto alla prima, risulta facilmente rilassabile usando il metodo lagrangiano [5], permettendo dunque di ottenere una funzione obiettivo separabile in sotto-problemi indipendenti, uno per ogni arco della rete.

Prima di procedere col rilassamento si definiscono $k \times i$ penalità lagrangiane: λ_i^k con k informazione e i nodo. Il numero corrisponde a quanti vincoli sono stati rilassati e in questo caso corrisponde al vincolo (2.9) che deve essere rispettato per ogni possibile combinazione di i e k .

Si riscrive il vincolo nella forma di uguaglianza a zero:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k = 0 \quad (2.14)$$

Questo viene successivamente incorporato nella funzione obiettivo insieme alle penalità:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i \in N} \lambda_i^k \left(\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k \right) \quad (2.15)$$

(2.15): La doppia sommatoria $\sum_{k \in K} \sum_{i \in N}$ è necessaria perché si vuole trovare il minimo della funzione considerando che il vincolo rilassato venga rispettato su tutte le combinazioni di k e i come da esso definito (2.9).

(2.15): Il + prima delle sommatorie è una scelta di comodità. In caso di violazione del vincolo, il segno di λ si adatta al segno del termine $(\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k)$ usando il metodo del subgradiente. Ciò tenderà a rendere il prodotto tra i due positivo e allontanerà la soluzione dal minimo, penalizzando il mancato rispetto del vincolo.

Si ha quindi:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lt_i} x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lr_i} x_{ji}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.16)$$

↓

$$\begin{aligned}
L(\lambda) = \min & \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k \\
& - \sum_{k \in K} \sum_{j,i \in A} \lambda_i^k x_{ji}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k
\end{aligned} \tag{2.17}$$

(2.16 $\rightarrow$ 2.17): La trasformazione delle sommatorie da $\sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lt_i} x_{ij}^k$ a $\sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k$ viene effettuata applicando prima la proprietà della linearità della sommatoria: $\sum_{k \in K} \sum_{i \in N} \sum_{j \in Lt_i} \lambda_i^k x_{ij}^k$ poi, notando che la doppia sommatoria $\sum_{i \in N} \sum_{j \in Lt_i}$ considera esattamente tutti gli archi i, j della rete, trascrivendola sostituendo $\sum_{i \in N} \sum_{j \in Lt_i}$ con $\sum_{i,j \in A}$.

Successivamente, il termine $\sum_{k \in K} \sum_{j,i \in A} \lambda_i^k x_{ji}^k$ viene rinominato cambiando $j \rightarrow i$ e $i \rightarrow j$:

$$\sum_{k \in K} \sum_{i,j \in A} \lambda_j^k x_{ij}^k \tag{2.18}$$

Ne consegue:

$$\begin{aligned}
L(\lambda) = \min & \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k \\
& - \sum_{k \in K} \sum_{i,j \in A} \lambda_j^k x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k
\end{aligned} \tag{2.19}$$

$$\begin{aligned}
& \downarrow \\
L(\lambda) = \min & \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} (\lambda_i^k - \lambda_j^k) x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k
\end{aligned} \tag{2.20}$$

$$\begin{aligned}
& \downarrow \\
L(\lambda) = \min & \sum_{k \in K} \sum_{i,j \in A} (c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k
\end{aligned} \tag{2.21}$$

(2.19) $\rightarrow$ (2.21): Nel passaggio tra le equazioni sono state svolte due semplici operazioni: un raccoglimento dei termini comuni e l'applicazione della proprietà della linearità.

Dato che l'ultimo termine non ha variabili decisionali può essere estratto in quanto nella ricerca del minimo non influenza il risultato.

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \tag{2.22}$$

La funzione obiettivo risulta ora separabile: per ogni arco i, j si ottiene un sotto-problema indipendente definito LR .

$$LR_{i,j \in A}(\lambda) = \min \sum_{k \in K} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \tag{2.23}$$

$$L(\lambda) = \sum_{i,j \in A} LR_{i,j}(\lambda) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.24)$$

$LR_{i,j}(\lambda)$ indica la soluzione ottima del sotto-problema per l'arco su cui è definita. Per la sua risoluzione si introduce una variabile d_{ij}^k che quantifica il costo dell'attivazione dell'arco i, j per l'informazione k :

$$\min c_{ij}e_{ij}^k + (\lambda_i^k - \lambda_j^k)x_{ij}^k \quad (2.25)$$

Si presentano due casi: se l'arco è disattivato, il costo è pari a 0, poiché $e_{ij}^k = 0$ implica $x_{ij}^k = 0$ per il vincolo (2.11). Se invece è attivato, d_{ij}^k rappresenta il costo dell'attivazione. Nel caso in cui d_{ij}^k sia minore di 0 conviene attivare l'arco in quanto si ha un guadagno.

Per calcolare d_{ij}^k si hanno già tutti i valori necessari: c_{ij} è noto, si pone $e_{ij}^k = 1$ poiché si vuole confrontare se si ha un guadagno dall'attivazione dell'arco e x_{ij}^k dipende da $(\lambda_i^k - \lambda_j^k)$. Poiché l'obiettivo è minimizzare, se il termine $(\lambda_i^k - \lambda_j^k)$ fosse positivo allontanerebbe dalla soluzione ottima e quindi x_{ij}^k assume il valore minimo consentito dal vincolo (2.11). Se invece fosse negativo contribuirebbe alla minimizzazione e quindi x_{ij}^k assume il valore massimo consentito che è pari a $NodiTarget_k$.

Determinati quali archi attivare e per quali informazioni, si incontra un ulteriore problema legato alla capacità degli archi: non tutte le informazioni selezionate potrebbero essere trasmesse simultaneamente all'interno dell'arco senza superare la sua capacità massima. Questo problema è riconducibile al classico problema del Knapsack [15] in cui l'arco rappresenta lo zaino con capacità u_{ij} e le informazioni k sono gli oggetti con peso a_k e valore $-d_{ij}^k$ che si vuole inserire all'interno. Poiché il Knapsack massimizza il valore degli oggetti inseriti e il guadagno è rappresentato in negativo da d_{ij}^k , invertendone il segno si rendono compatibili i due problemi.

2.5.1 Aggiornamento delle Penalità Lagrangiane

Per l'aggiornamento delle penalità si utilizza il metodo del subgradiente [5] come descritto nella sezione (1.4.2).

Si prende il vincolo rilassato e da esso si definisce il subgradiente:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k = 0 \quad (2.26)$$

↓

$$s_i^k = \sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k \quad (2.27)$$

La variabile s_i^k rappresenta quanto il vincolo rilassato (2.9) viene violato nella soluzione corrente e dipende da k e i poiché il vincolo stesso è definito per ogni combinazione di informazione k e nodo i . Definito il subgradiente, lo si calcola insieme alle variabili x_{ij}^k e si aggiornano le penalità λ_i^k secondo la formula $\lambda_i^k = \lambda_i^k + \theta s_i^k$.

Per scegliere θ si utilizza l'equazione:

$$\theta = \alpha \frac{\text{UpperBound} - \text{LowerBound}}{\|s\|^2} \quad (2.28)$$

I parametri sono scelti in modo da garantire passi ampi in caso di lontananza dall'ottimo, favorendo una convergenza rapida, e passi più piccoli nelle iterazioni vicine all'ottimo, evitando la divergenza dalla soluzione.

- α è un moltiplicatore che viene dimezzato ogni volta che il LowerBound non migliora per un numero prefissato di iterazioni. Se il LowerBound non migliora è probabile si stia lavorando con passi troppo elevati.
- $\text{UpperBound} - \text{LowerBound}$ rappresenta l'ampiezza dell'intervallo in cui si trova la soluzione ottima. Se i due bound sono distanti si consiglia di effettuare passi più grandi mentre al contrario bisogna lavorare con passi più piccoli e una maggiore precisione.
- $\|s\|^2$ è la norma al quadrato del vettore dei subgradienti. Un valore elevato indica una violazione significativa del vincolo e dividendo per esso si riduce il passo, evitando aggiornamenti eccessivi di λ . Al contrario, una norma di valore piccolo indica che si è vicini all'ottimo e quindi dividendo per esso si aumenta il passo evitando una convergenza troppo lenta.

3 Miglioramento della Formulazione Rispetto ai Percorsi Parziali

3.1 Nuova Formulazione

Le formulazioni descritte in precedenza presentano un limite nel fondamento logico: i vincoli (2.2) e (2.9) impongono che ogni Subscriber riceva l'informazione richiesta. Di conseguenza, se anche un solo destinatario non è raggiungibile, tali vincoli risultano violati, il problema non ammette soluzioni e non è possibile perciò calcolare una soluzione parziale, ovvero una in cui solo un sottoinsieme di riceventi viene soddisfatto. Per superare questa limitazione si introduce una nuova variabile decisionale binaria $y_i^k \in \{0, 1\}$ che indica se un determinato nodo i va servito con l'informazione k . Questa variabile ricopre un ruolo chiave in due aspetti del problema: la funzione obiettivo del problema e la definizione di b_i^k .

Per la sua integrazione si aggiorna la seconda formulazione e la definizione di b_i^k :

$$\text{minimizza } \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + P \sum_{k \in K} \sum_{t \in \text{NodiTarget}_k} (1 - y_t^k) \quad (3.1)$$

dove P è la penalità per la mancata consegna a un nodo richiedente.

$$b_i^k = \begin{cases} \sum_{t \in \text{NodiTarget}_k} y_t^k & \text{se } i \text{ Publisher} \\ -y_i^k & \text{se } i \text{ Subscriber} \\ 0 & \text{se } i \text{ Broker} \end{cases} \quad (3.2)$$

P è scelto con un valore sufficientemente grande affinché, se un nodo è raggiungibile, risulti sempre più conveniente servirlo piuttosto che pagare la penalità. Idealmente si può pensare a P come il costo complessivo di tutti gli archi della rete sommato a uno. In questo modo, anche un percorso che utilizza tutti gli archi risulta preferibile al mancato servizio del nodo.

$$P = \left(\sum_{i,j \in A} c_{ij} \right) + 1 \quad (3.3)$$

Per decidere quali nodi servire e quali no, si applica lo stesso procedimento basato sul rilassamento lagrangiano già eseguito in precedenza per le variabili x e e . Si riprende dunque il lagrangiano ottenuto al punto 2.5 (equazione (2.21)) e vi si aggiunge il nuovo termine della funzione obiettivo in quanto non altera i calcoli già effettuati:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} (c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k + P \sum_{k \in K} \sum_{t \in \text{NodiTarget}_k} (1 - y_t^k) \quad (3.4)$$

Si isola dunque la seconda parte dell'equazione, contenente la nuova variabile, per ottenere la condizione di attivazione di y :

$$L(\lambda) = \min P \sum_{k \in K} \sum_{t \in NodiTarget_k} (1 - y_t^k) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (3.5)$$

Si separano i nodi nelle diverse tipologie e si sviluppano i prodotti con le corrispondenti λ :

$$L(\lambda) = \min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{k \in K} \left(\sum_{br \in Brokers} \lambda_{br}^k b_{br}^k + \sum_{o \in NodoOrigine_k} \lambda_o^k b_o^k + \sum_{t \in NodiTarget_k} \lambda_t^k b_t^k \right) \quad (3.6)$$

$$\downarrow$$

$$L(\lambda) = \min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{k \in K} \left(\sum_{o \in NodoOrigine_k} \lambda_o^k y_t^k + \sum_{t \in NodiTarget_k} -\lambda_t^k y_t^k \right) \quad (3.7)$$

(3.6)→(3.7): Nel passaggio tra le due equazioni, si sostituiscono le variabili b con i valori stabiliti dalla definizione.

$$L(\lambda) = \min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{k \in K} \sum_{t \in NodiTarget_k} (y_t^k (\lambda_o^k - \lambda_t^k)) \quad (3.8)$$

(3.8): La sommatoria sui nodi di origine viene rimossa perché, per ogni k , esiste un unico nodo che genera l'informazione.

$$L(\lambda) = \min \sum_{k \in K} \sum_{t \in NodiTarget_k} (P - P y_t^k) - y_t^k (\lambda_o^k - \lambda_t^k) \quad (3.9)$$

$$\downarrow$$

$$L(\lambda) = \min \sum_{k \in K} \sum_{t \in NodiTarget_k} P - y_t^k (\lambda_o^k - \lambda_t^k + P) \quad (3.10)$$

La condizione di attivazione di y_t^k si ottiene confrontando il costo nel caso in cui il nodo target t sia servito ($y_t^k = 1$) rispetto al caso in cui non lo sia ($y_t^k = 0$). Nel momento in cui il costo associato col caso servito risulta minore si ha un guadagno e quindi si attiva y_t^k :

$$P - \lambda_o^k + \lambda_t^k - P < P \rightarrow \lambda_t^k - \lambda_o^k < P \quad (3.11)$$

Pertanto $y_t^k = 1$ quando $\lambda_t^k - \lambda_o^k < P$, con t nodo target e o nodo origine di k .

3.2 Mancata Convergenza del Lower Bound alla Soluzione Intera

La nuova formulazione presenta una criticità sottile ma rilevante: il vincolo (2.11) modificato si configura come un classico esempio di formulazione debole (1.4.3):

$$e_{ij}^k \leq x_{ij}^k \leq e_{ij}^k \sum_{t \in \text{NodiTarget}_k} y_t^k \quad \forall k \in K, (i, j) \in A \quad (3.12)$$

La presenza del fattore $\sum_{t \in \text{NodiTarget}_k} y_t^k$ è la causa diretta di tale debolezza.

3.2.1 Analisi del Problema nel Dettaglio

Il vincolo lega la variabile x_{ij}^k alla variabile binaria di attivazione e_{ij}^k . Seguendo la logica del vincolo ci si aspetterebbe che se non transita alcuna informazione ($x = 0$) allora il vincolo imponga $e = 0$, mentre nel caso contrario costringa $e = 1$. Nella realtà la presenza del fattore $\sum_{t \in \text{NodiTarget}_k} y_t^k$ altera questa dinamica. Prendendo come esempio una rete con un'informazione k che deve raggiungere due target distinti t_1 e t_2 , negli archi in cui la variabile x assume valore singolo ($x_{ij}^k = 1$), il vincolo esprime $1 \leq 2e$, ovvero $e \geq \frac{1}{2}$. Nel rilassamento continuo alla variabile e sarà assegnato il valore minore possibile per minimizzare la funzione obiettivo e quindi assumerà valore 0.5. Nonostante il risolutore costringa la variabile ad assumere valore intero, e quindi 1, il valore frazionario rimarrà impresso nelle λ che guidano l'algoritmo durante le iterazioni successive. L'effetto pratico che si osserva è il seguente: l'algoritmo tende a penalizzare l'arco attivo e a favorire quello disattivo, per poi invertire, mantenendo così una media di attivazione pari a 0.5. Questa oscillazione impedisce al Lower Bound di crescere oltre una certa soglia.

3.2.2 Scrittura in Formulazione Forte

Per eliminare la debolezza del vincolo, si ridefiniscono le variabili x , b e λ in modo da distinguere il nodo target che l'informazione deve raggiungere. Nascono così le variabili x_{ij}^{kt} , b_i^{kt} e λ_i^{kt} .

I nuovi vincoli derivanti sono:

$$x_{ij}^{kt} \leq e_{ij}^k \quad \forall k \in K, t \in \text{NodiTarget}_k, (i, j) \in A \quad (3.13)$$

$$\sum_{j \in Lt_i} x_{ij}^{kt} - \sum_{j \in Lr_i} x_{ji}^{kt} = b_i^{kt} \quad \forall i \in N, t \in \text{NodiTarget}_k, k \in K \quad (3.14)$$

La definizione di b_i^{kt} diventa:

$$b_i^{kt} = \begin{cases} y_i^k & \text{se } i \text{ Publisher dell'informazione } k \text{ diretta al target } t \\ -y_i^k & \text{se } i \text{ Subscriber dell'informazione } k \text{ e } i = t \\ 0 & \text{se } i \text{ Broker} \end{cases} \quad (3.15)$$

Con questa formulazione, l'arco viene attivato non appena dell'informazione lo attraversa, per qualsiasi target, ma il legame rimane diretto. In questo modo, se l'arco è disattivato l'informazione passante è nulla e, viceversa, se attivo è limitata a uno per target (3.13). Ciò toglie la possibilità di soluzioni frazionarie e rimuove le oscillazioni descritte in precedenza.

3.2.3 Nuove Condizioni di Attivazione

A seguito della ridefinizione delle variabili decisionali, occorre riformulare le condizioni di scelta per l'attivazione degli archi, l'instradamento delle informazioni e la selezione dei nodi da servire.

Tali condizioni vengono ricalcolate seguendo il medesimo procedimento di rilassamento lagrangiano descritto in precedenza (2.5 e 3.1):

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + P \sum_{k \in K} \sum_{t \in \text{NodiTarget}_k} (1 - y_t^k) + \sum_{k \in K} \sum_{t \in \text{NodiTarget}_k} \sum_{i \in N} \lambda_i^{kt} \left(\sum_{j \in Lt_i} x_{ij}^{kt} - \sum_{j \in Lr_i} x_{ji}^{kt} - b_i^{kt} \right) \quad (3.16)$$

↓

$$L(\lambda) = \min \sum_{k \in K} \left(\sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{t \in \text{NodiTarget}_k} \left(\sum_{i,j \in A} \lambda_i^{kt} x_{ij}^{kt} - \sum_{j,i \in A} \lambda_i^{kt} x_{ji}^{kt} - \left(\sum_{o \in \text{NodoOrigine}_{kt}} \lambda_o^{kt} y_t^k + \sum_{ta \in \text{NodiTarget}_{kt}} -\lambda_{ta}^{kt} y_t^k \right) \right) + \sum_{t \in \text{NodiTarget}_k} P(1 - y_t^k) \right) \quad (3.17)$$

Essendo unici il nodo di origine o e il nodo target ta , per ogni informazione k e target t , le rispettive sommatorie presentano un unico termine, e possono quindi essere rimosse. Inoltre, poiché ta e t indicano lo stesso nodo, è possibile sostituire ta con t :

$$L(\lambda) = \min \sum_{k \in K} \left(\sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{t \in \text{NodiTarget}_k} \sum_{i,j \in A} (\lambda_i^{kt} - \lambda_j^{kt}) x_{ij}^{kt} + \sum_{t \in \text{NodiTarget}_k} (P - P y_t^k) - \sum_{t \in \text{NodiTarget}_k} (\lambda_o^{kt} - \lambda_t^{kt}) y_t^k \right) \quad (3.18)$$

La funzione obiettivo può essere suddivisa in due sotto-problemi indipendenti, ciascuno utilizzato per ricavare le condizioni di attivazione delle relative variabili decisionali al loro interno.

Il primo sotto-problema permette di ricavare le condizioni per e_{ij}^k e x_{ij}^{kt} :

$$\min \sum_{k \in K} \sum_{i,j \in A} (c_{ij} e_{ij}^k + \sum_{t \in NodiTarget} (\lambda_i^{kt} - \lambda_j^{kt}) x_{ij}^{kt}) \quad (3.19)$$

Per attivare e_{ij}^k si confronta il caso attivo con quello disattivo, mirando a una riduzione del valore della funzione obiettivo:

$$c_{ij} + \sum_{t \in NodiTarget} (\lambda_i^{kt} - \lambda_j^{kt}) x_{ij}^{kt} < 0 \quad (3.20)$$

Nello specifico, se $(\lambda_i^{kt} - \lambda_j^{kt}) < 0$, si pone $x_{ij}^{kt} = 1$ altrimenti $x_{ij}^{kt} = 0$.
Il secondo sotto-problema permette di ricavare la condizione per y_t^k :

$$\min \sum_{k \in K} (\sum_{t \in NodiTarget_k} (P - P y_t^k) - \sum_{t \in NodiTarget} (\lambda_o^{kt} - \lambda_t^{kt}) y_t^k) \quad (3.21)$$

Lo sviluppo dell'equazione porta alla condizione descritta in precedenza (3.11), rimasta invariata: $\lambda_t^{kt} - \lambda_o^{kt} < P$.

3.2.4 Terza Formulazione Completa

Funzione Obiettivo:

$$\text{minimizza} \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + P \sum_{k \in K} \sum_{t \in NodiTarget_k} (1 - y_t^k) \quad (3.22)$$

Vincoli:

$$\sum_{j \in Lt_i} x_{ij}^{kt} - \sum_{j \in Lr_i} x_{ji}^{kt} = b_i^{kt} \quad \forall i \in N, t \in NodiTarget_k, k \in K \quad (3.23)$$

$$\sum_{k \in K} a^k e_{ij}^k \leq u_{ij} \quad \forall (i, j) \in A \quad (3.24)$$

$$x_{ij}^{kt} \leq e_{ij}^k \quad \forall k \in K, t \in NodiTarget_k, (i, j) \in A \quad (3.25)$$

$$x_{ij}^{kt} \geq 0 \quad \forall (i, j) \in A, k \in K \quad (3.26)$$

$$e_{ij}^k \in \{0, 1\} \quad \forall (i, j) \in A, k \in K \quad (3.27)$$

$$y_t^k \in \{0, 1\} \quad \forall k \in K, t \in NodiTarget_k \quad (3.28)$$

4 Implementazione Algoritmica

In questo capitolo vengono presentati e descritti gli algoritmi sviluppati sulla base logica delle formulazioni analizzate nel capitolo precedente, illustrandone il funzionamento strutturato su due fasi.

Nella prima fase si calcola un Upper Bound, ovvero una soluzione ammissibile del problema non necessariamente ottimale, mediante metodi euristici, con l'obiettivo di contenere i tempi di calcolo che, altrimenti, risulterebbero molto elevati.

Nella seconda fase, tale valore viene utilizzato come riferimento per guidare la ricerca del miglior Lower Bound, sfruttando l'aggiornamento delle penalità lagrangiane descritto in precedenza (2.5.1).

Il processo, infine, si conclude con un confronto volto a valutare la qualità delle soluzioni ottenute e la loro distanza dall'ottimo.

Le principali classi di algoritmi implementate sono:

- Algoritmo Cplex: il modello matematico viene direttamente implementato nell'ambiente di sviluppo di Cplex [10] o in Python. La soluzione ottima restituita costituisce il riferimento per il confronto con i risultati degli altri algoritmi.
- Algoritmo Lagrangiano: implementato in Python, applica la logica teorica del rilassamento lagrangiano [5] e delle condizioni di attivazione per il calcolo del Lower Bound ottimo.
- Algoritmo euristico per l'Upper Bound: ispirato alla logica di Dijkstra [7], che viene utilizzato per la ricerca rapida di un Upper Bound ammissibile.

Ogni algoritmo presentato in questo capitolo utilizza le variabili già definite nel paragrafo dei simboli globali (2.2).

4.1 Implementazione delle Formulazioni Senza Soluzioni Parziali

In questa sezione si analizzano le due formulazioni che non prevedono la possibilità di generare soluzioni parziali insieme al loro algoritmo per la ricerca di un Upper Bound ammissibile.

4.1.1 Upper Bound Euristico

Per il calcolo dell'Upper Bound relativo alla prima e alla seconda formulazione, si è optato per lo sviluppo di un algoritmo euristico basato sulla logica di Dijkstra per i cammini minimi.

Poiché l'ordine di instradamento delle informazioni influenza significativamente

l'allocazione delle capacità sugli archi e la qualità della soluzione finale, l'algoritmo adotta un approccio Multi-Start ripetuto per un numero arbitrario di iterazioni. A ogni iterazione, l'insieme delle informazioni viene mescolato casualmente e, per ciascuna di esse, viene eseguita una variante dell'algoritmo di Dijkstra che integra un controllo preventivo sulle capacità residue degli archi. Tale vincolo impone che un nodo adiacente venga preso in considerazione solo se l'arco che lo raggiunge è in grado di sostenere il peso dell'informazione corrente. Al termine di ogni iterazione, i cammini ottimi vengono ricostruiti a ritroso a partire dai nodi target e le capacità residue della rete vengono aggiornate di conseguenza. L'algoritmo, infine, restituisce i percorsi che hanno il costo complessivo minore.

Le variabili in ingresso coincidono con quelle definite nel paragrafo (2.2).

Nella prima parte di codice si definiscono le strutture globali utilizzate per memorizzare il cammino ottimo e il corrispondente costo.

```
percorsoOttimo=[]  
costoTot = np.inf
```

All'inizio di ogni iterazione dell'algoritmo viene creata una copia delle strutture necessarie per i calcoli e viene mescolato l'ordine di elaborazione delle informazioni.

```
costo = 0.0  
strade = A.copy()  
CapacitàUsata = u.copy()  
informazioni = K.copy()  
random.shuffle(informazioni)  
percorsi = []  
soluzioneValida = True
```

L'algoritmo procede iterando su ogni informazione e, a ogni iterazione, determina l'insieme di nodi da visitare filtrati per raggiungibilità. Nello specifico, qualora per un dato nodo g l'insieme dei predecessori risulti vuoto, esso viene rimosso.

```
if not soluzioneValida:  
    break  
unvisitedNodes = [i for i in N]  
for g,l in Lr.items():  
    if len(l)==0:  
        unvisitedNodes.remove(g)  
nodoDiPartenza = NodoOrigine[informazione]  
if nodoDiPartenza not in unvisitedNodes:  
    unvisitedNodes.append(nodoDiPartenza)
```

Successivamente viene applicata una variante dell'algoritmo di Dijkstra con controllo sulle capacità per determinare l'albero dei cammini minimi. Per l'elaborazione viene utilizzata una tabella in grado di memorizzare il costo del cammino minimo temporaneo e il relativo nodo predecessore.

```
while (len(unvisitedNodes)>0):
```

```

for (i,j) in strade:
    if (i==nodoProtagonista and j in unvisitedNodes):
        if CapacitàUsata[(i, j)] - a[informazione] >= 0:
            if (tabella[j][1]> tabella[i][1]+C[i, j]):
                tabella[j][1]= tabella[i][1]+C[i, j]
                tabella[j][2]= nodoProtagonista
unvisitedNodes.remove(nodoProtagonista)
indiceProssimoNodo = -1
for j in unvisitedNodes:
    if tabella[j][1]<math.inf:
        if indiceProssimoNodo == -1:
            indiceProssimoNodo = j
        else:
            if (tabella[j][1]< tabella[indiceProssimoNodo][1]):
                indiceProssimoNodo = j

```

In caso di nodi non raggiungibili l'algoritmo viene fermato forzatamente altrimenti si prosegue con la sua esecuzione.

```

if indiceProssimoNodo==-1:
    unvisitedNodes=[]
else:
    nodoProtagonista = indiceProssimoNodo

```

Completata l'esplorazione della rete, i percorsi ottimi vengono ricostruiti a ritroso a partire dai rispettivi nodi target e viene aggiornata la capacità residua della rete. In caso di target non raggiunti l'iterazione dell'algoritmo viene considerata fallita.

```

nodiFinali = NodiTarget[informazione].copy()
random.shuffle(nodiFinali)
for nodoFinale in nodiFinali:
    percorso=[]
    percorso.append(nodoFinale)
    if (tabella[nodoFinale][2]==None):
        costo=np.inf
        soluzioneValida = False
        break
    while (tabella[nodoFinale][2]!=None):
        preFinale= tabella[nodoFinale][2]
        percorso.append(preFinale)
        CapacitàUsata[(preFinale, nodoFinale)]= CapacitàUsata[(preFinale, nodoFinale)]-a[informazione]
        costo += C[preFinale, nodoFinale]
        nodoFinale = preFinale
    percorsi.append(informazione)
    percorsi.append(percorso)

```

Al termine del ciclo di controllo su tutte le informazioni, se la soluzione è ammissibile, viene confrontata con la precedente migliore e, qualora quest'ultima risulti

peggiore in termini di costo, viene sostituita.

```
if soluzioneValida and costo < costoTot:
    costoTot = costo
    percorsoOttimo = percorsi
```

4.1.2 Prima Formulazione

Per quanto riguarda la prima formulazione, lo sviluppo si è concentrato sulla creazione del modello in ambiente di sviluppo Cplex e in Python escludendo l'implementazione della metodologia basata sul rilassamento lagrangiano.

Cplex

Nell'ambiente Cplex il codice risulta molto lineare, con una distinzione chiara tra la funzione obiettivo e i vincoli.

```
minimize
sum(k in K, i in N, j in Lr[i]) C[<j,i>] * x[k][<j,i>];
subject to {
    forall(k in K, t in NodiTarget[k])
        sum(j in Lr[t]) x[k][<j,t>] == 1;

    forall(k in K, i in Brokers)
        sum(j in Lr[i]) x[k][<j,i>] == sum(j in Lt[i]) x[k][<i,j>];

    forall(k in K, <i,j> in A)
        bigM*e[k][<i,j>] >= x[k][<i,j>];

    forall(i in N, j in Lt[i])
        sum(k in K) a[k]*(e[k][<i,j>]+e[k][<j,i>]) <= u[<i,j>];
}
```

Python API

Nel codice in Python la scrittura si complica e le definizioni si fanno più complesse. Inizialmente vengono definite e generate le variabili decisionali con i rispettivi vincoli associati: (2.6) e (2.7).

```
modello = cplex.Cplex()
modello.objective.set_sense(modello.objective.sense.minimize)
x = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"x_{k}_{i}_{j}" for (k,i,j) in x],
lb=[0]*len(x),
```

```

types=["I"]*len(x))
e = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"e_{k}_{i}_{j}" for (k,i,j) in e],
lb=[0]*len(e),
ub=[1]*len(e),
types=["B"]*len(e))

```

Generate le variabili decisionali si procede con la definizione della funzione obiettivo e dei vincoli del problema.

Funzione Obiettivo (2.1):

```

temp_X = []
temp_Coeffs=[]
for k in K:
    for i in N:
        for j in Lr[i]:
            if (j,i) in A:
                temp_Coeffs.append(C[j,i])
                temp_X.append(f"x_{k}_{j}_{i}")
modello.objective.set_linear(list(zip(temp_X,temp_Coeffs)))

```

Primo vincolo (2.2):

```

for k in K:
    for t in NodiTarget[k]:
        temp_X = []
        temp_Coeffs=[]
        for j in Lr[t]:
            if (j,t) in A:
                temp_X.append(f"x_{k}_{j}_{t}")
                temp_Coeffs.append(1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X,temp_Coeffs)],senses=["E"],rhs=[1])

```

Secondo vincolo (2.3):

```

for k in K:
    for i in Brokers:
        temp_X=[]
        temp_Coeffs=[]
        for j in Lr[i]:
            if (j,i) in A:
                temp_X.append(f"x_{k}_{j}_{i}")
                temp_Coeffs.append(1)
        for j in Lt[i]:
            if (i,j) in A:
                temp_X.append(f"x_{k}_{i}_{j}")
                temp_Coeffs.append(-1)
        modello.linear_constraints.add

```

```
(lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["E"], rhs=[0])
```

Terzo vincolo (2.4):

```
for k in K:
    for (i, j) in A:
        temp_X=[]
        temp_Coeffs=[]
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(bigM)
        temp_X.append(f"x_{k}_{i}_{j}")
        temp_Coeffs.append(-1)
        modello.linear_constraints.add
(lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["G"], rhs=[0])
```

Quarto vincolo (2.5):

```
for i in N:
    for j in Lt[i]:
        temp_X=[]
        temp_Coeffs=[]
        for k in K:
            if (i, j) in A:
                temp_X.append(f"e_{k}_{i}_{j}")
                temp_Coeffs.append(a[k])
            if (j, i) in A:
                temp_X.append(f"e_{k}_{j}_{i}")
                temp_Coeffs.append(a[k])
        modello.linear_constraints.add
(lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[u[i, j]])
```

4.1.3 Seconda Formulazione

La seconda formulazione, oltre all'implementazione nell'ambiente Cplex e in Python tramite API, prevede lo sviluppo di un algoritmo basato sulla teoria del rilassamento lagrangiano strutturato in diverse funzioni, ognuna con il proprio specifico ruolo.

Calcolo Costi Penalizzati

Il calcolo dei costi penalizzati si basa sull'equazione (2.25), la quale quantifica il guadagno associato all'attivazione di un determinato arco. Qualora tale operazione determini una riduzione del valore della funzione obiettivo, l'arco viene inserito all'interno di una lista di candidati per una futura elaborazione.

```
for (k, i, j) in d:
    if (lam[(k, i)] - lam[(k, j)]) < 0:
```

```

        d[(k, i, j)] = C[i, j] + ((lam[(k, i)] - lam[(k, j)]) * len(NodiTarget[k]))
    else:
        d[(k, i, j)] = C[i, j]
    if (d[(k, i, j)] < 0):
        ePerKnapsack.append((k, i, j))

```

KnapSack

Per garantire il rispetto del vincolo sulla capacità degli archi, si è optato per lo sviluppo di un algoritmo basato sulla programmazione dinamica applicata al problema dello zaino (KnapSack problem) [15]. Il problema dello zaino è molto famoso nel panorama della ricerca operativa e il suo scopo è quello di inserire la migliore combinazione di oggetti possibile all'interno di uno zaino dotato di una capacità massima limitata. Ogni oggetto è caratterizzato da un peso e da un guadagno, di conseguenza, l'obiettivo finale è la massimizzazione del profitto totale. Trattandosi di un problema complesso, i tempi di risoluzione risultano molto elevati con metodi tradizionali. Per superare questo limite, si sfrutta la programmazione dinamica, ovvero si suddivide in sotto-problemi e, per il calcolo di ciascuno di essi, si sfruttano le operazioni già effettuate in quelli precedenti al fine di ridurre i tempi. La capacità massima dello zaino coincide con la capacità dell'arco, il peso degli oggetti da inserire è rappresentato dalla banda occupata dall'informazione e il profitto è dato dal valore opposto del costo penalizzato, che viene reso positivo per consentirne l'utilizzo diretto. In caso di attivazione dell'arco la relativa variabile x verrà aggiornata seguendo la logica espressa nel paragrafo (2.5).

In primo luogo, a ciascun arco vengono associate le informazioni che si ritengono candidate a transitarvi.

```

kPerArco = {}
for (k, i, j) in ePerKnapsack:
    if (i, j) not in kPerArco:
        kPerArco[(i, j)] = []
        kPerArco[(i, j)].append(k)

```

Successivamente si esegue l'algoritmo dello zaino per ogni arco, inserendo i parametri nelle apposite strutture dati.

```

for (i, j) in kPerArco.keys():
    valoriD = []
    peso = u[(i, j)]
    pesi = []
    valoriD.append(0)
    pesi.append(0)
    listaK = []
    listaK.append(None)
    for k in kPerArco[(i, j)]:
        valoriD.append(-1*d[(k, i, j)])
        pesi.append(a[k])
        listaK.append(k)

```

Si procede quindi alla generazione della matrice di programmazione dinamica, all'interno della quale vengono memorizzati i valori ottimi dei relativi sotto-problemi, suddivisi in base alla capacità dello zaino e agli oggetti selezionabili.

```

tabella = []
for z in range(len(valoriD)):
    riga = []
    for y in range(peso+1):
        riga.append(0)
    tabella.append(riga)
for z in range(1, len(valoriD)):
    for y in range(1, peso+1):
        if (y >= pesi[z]):
            tabella[z][y] = max(tabella[z-1][y], tabella[z-1][y-pesi[z]] + valoriD[z])
        else:
            tabella[z][y] = tabella[z-1][y]

```

Finito il riempimento della tabella, la soluzione ottima è identificata dalla cella in basso a destra nella matrice. Partendo da essa, si esegue un controllo a ritroso per recuperare le informazioni scelte da far passare e aggiornare le variabili e e x .

```

migliore = tabella[len(valoriD)-1][peso]
rigaDelMigliore = len(valoriD)-1
colonnaDelMigliore = peso
while (rigaDelMigliore > 0):
    if (migliore != tabella[rigaDelMigliore-1][colonnaDelMigliore]):
        k = listaK[rigaDelMigliore]
        e[(k,i,j)] = 1.0
        if (lam[k,i]-lam[k,j] >= 0):
            x[(k,i,j)] = 1
        else:
            x[(k,i,j)] = len(NodiTarget[k])
        colonnaDelMigliore = colonnaDelMigliore - pesi[rigaDelMigliore]
        rigaDelMigliore = rigaDelMigliore - 1
        migliore = tabella[rigaDelMigliore][colonnaDelMigliore]
    else:
        rigaDelMigliore = rigaDelMigliore - 1

```

Calcola Step

Per determinare l'importanza dell'aggiornamento da applicare alle penalità lagrangiane, viene calcolato lo step del subgradiente con la formula descritta (2.28).

```

listaSubGradienti = []
for k in K:
    for i in N:
        listaSubGradienti.append(subGradient[(k,i)])
norma = np.linalg.norm(listaSubGradienti)

```

```

norma = norma.item()
if norma!=0:
    valoreDiCambio = alpha*((Ub-lastLb[-1])/(norma**2))
else:
    valoreDiCambio = alpha*((Ub-lastLb[-1]))

```

Calcolo del Lower Bound attuale

Con l'obiettivo di monitorare la convergenza dell'algoritmo e di aggiornare lo step, il valore del Lower Bound viene ricalcolato a ogni iterazione eseguendo l'operazione della funzione obiettivo lagrangiana (2.20).

```

Lb=0.0
for (i,j) in A:
    for k in K:
        Lb += d[(k,i,j)]*e[(k,i,j)]
for k in K:
    for i in N:
        Lb -= lam[(k,i)]*b[k][i]

```

Aggiornamento Penalità Lagrangiane

Sfruttando lo step precedentemente calcolato, si procede all'aggiornamento delle penalità lagrangiane secondo la formula $\lambda_i^k = \lambda_i^k + \theta s_i^k$.

```

subGradienti = {(k,i):0.0 for k in K for i in N}
for k in K:
    for i in N:
        Somatica1 = 0.0
        Somatica2 = 0.0
        for j in Lt[i]:
            Somatica1 += x[(k,i,j)]
        for j in Lr[i]:
            Somatica2 += x[(k,j,i)]
        subGradienti[(k,i)]=Somatica1-Somatica2-b[k][i]
valoreDiCambio = calcolaValoreDiCambio(lastLb ,subGradienti ,Ub, alpha)
for k in K:
    for i in N:
        lam[(k,i)] = lam[(k,i)] + valoreDiCambio *subGradienti[(k,i)]

```

Main Loop

Le funzioni appena elencate vengono gestite all'interno di un ciclo principale, il quale itera i metodi fino al raggiungimento del Lower Bound ottimo o all'esecuzione di un numero arbitrario di cicli. In questa implementazione, il parametro α

per la riduzione dello step viene dimezzato se non si incontra un miglioramento del Lower Bound per 20 cicli di fila, rispetto agli ultimi 20 calcolati. La scelta di tali parametri mira a trovare un equilibrio tra una frequenza di dimezzamento di α troppo elevata e una eccessivamente rara.

```

lam = {(k,i):0.0 for k in K for i in N}
d = {(k,i,j):0 for k in K for (i,j) in A}
e = {(k,i,j):0 for k in K for (i,j) in A}
x = {(k,i,j):0.0 for k in K for (i,j) in A}
ePerKnapsack = []
ultimiLowerBounds = [0]
Lb=0.0
alpha = 0.1
it_max=10000
alphaterazioni=0
iterazioni=0
Ub,percorso = DijkstraEuristico()
while abs(Ub-Lb)>1e-4 and iterazioni<it_max:
    iterazioni+=1
    ePerKnapsack=[]
    e = {(k,i,j):0 for k in K for (i,j) in A}
    x = {(k,i,j):0 for k in K for (i,j) in A}
    CalcolaCostiPenalizzati(lam,d,ePerKnapsack)
    KnapSack(ePerKnapsack,d,e,x)
    Lb = calcolaMin(lam)
    ultimiLowerBounds.append(Lb)
    if (ultimiLowerBounds[-1]<=max(ultimiLowerBounds[-21:-1])):
        alphaterazioni+=1
    if (alphaterazioni>20):
        alphaterazioni=0
        alpha=alpha/2
    else:
        alphaterazioni=0
    aggiornaLambda(lam,ultimiLowerBounds,Ub,alpha)

```

Cplex

Similmente alla formulazione precedente, l'ambiente Cplex consente una scrittura del codice più lineare e leggibile. In particolare, poiché il vincolo (2.9) ne incorpora due distinti (2.2) e (2.3), il modello finale risulta più compatto rispetto alla prima formulazione.

```

minimize
sum(k in K, <i,j> in A) C[<i,j>] * e[k][<i,j>];
subject to {
    forall(k in K, i in N : i != NodoOrigine[k])
        sum(j in Lt[i]) x[k][<i,j>] - sum(j in Lr[i]) x[k][<j,i>] == b[k][i];

```

```

forall(<i,j> in A)
sum(k in K) a[k] * e[k][<i,j>] <= u[<i,j>];

forall(k in K, <i,j> in A) {
    e[k][<i,j>] <= x[k][<i,j>];
    x[k][<i,j>] <= card(NodiTarget[k]) * e[k][<i,j>];
}
}

```

Python API

Per quanto riguarda l'implementazione in Python tramite le API di Cplex, il terzo vincolo deve essere suddiviso in due sotto-vincoli per garantire il rispetto di entrambe le condizioni.

Inizialmente, vengono generate le variabili decisionali con i relativi vincoli associati (2.12) e (2.13).

```

modello = cplex.Cplex()
modello.objective.set_sense(modello.objective.sense.minimize)
x = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"x_{k}_{i}_{j}" for (k,i,j) in x],
lb=[0]*len(x),
types=["I"]*len(x))
e = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"e_{k}_{i}_{j}" for (k,i,j) in e],
lb=[0]*len(e),
ub=[1]*len(e),
types=["B"]*len(e))

```

Istanziare le variabili, si procede con la definizione della funzione obiettivo e dei vincoli della formulazione. Funzione Obiettivo (2.8):

```

temp_X = []
temp_Coeffs=[]
for k in K:
    for (i,j) in A:
        temp_Coeffs.append(C[i,j])
        temp_X.append(f"e_{k}_{i}_{j}")
modello.objective.set_linear(list(zip(temp_X, temp_Coeffs)))

```

Primo vincolo (2.9):

```

for k in K:
    for i in N:
        if i != NodoOrigine[k]:
            temp_X = []
            temp_Coeffs=[]
            for j in Lt[i]:

```

```

        if (i, j) in A:
            temp_X.append(f"x_{k}_{i}_{j}")
            temp_Coeffs.append(1)
    for j in Lr[i]:
        if (j, i) in A:
            temp_X.append(f"x_{k}_{j}_{i}")
            temp_Coeffs.append(-1)
    modello.linear_constraints.add
    (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)],
    senses=["E"], rhs=[b[k][i]])

```

Secondo vincolo (2.10):

```

for (i, j) in A:
    temp_X = []
    temp_Coeffs = []
    for k in K:
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(a[k])
    modello.linear_constraints.add
    (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[u[i, j]])

```

Terzo vincolo (2.11):

```

for k in K:
    for (i, j) in A:
        temp_X = []
        temp_Coeffs = []
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(1)
        temp_X.append(f"x_{k}_{i}_{j}")
        temp_Coeffs.append(-1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[0])

for k in K:
    for (i, j) in A:
        temp_X = []
        temp_Coeffs = []
        temp_X.append(f"x_{k}_{i}_{j}")
        temp_Coeffs.append(1)
        temp_X.append(f"e_{k}_{i}_{j}")
        temp_Coeffs.append(-len(NodiTarget[k]))
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[0])

```

4.2 Formulazione Con Soluzioni Parziali

In questa sezione si analizza la formulazione che prevede la possibilità di generare soluzioni parziali insieme al suo algoritmo per la ricerca di un Upper Bound ammissibile.

4.2.1 Dijkstra a Scenari e Meccanismo di Discesa dell'Upper Bound

Per quanto riguarda la nuova formulazione, l'approccio alla ricerca di un Upper Bound ammissibile è stato radicalmente cambiato per superare i limiti della precedente versione. Nel modello antecedente l'algoritmo, l'impossibilità di raggiungere anche un solo nodo target da parte della rispettiva informazione decretava il fallimento dell'intera esecuzione. Nella nuova versione, divisa in funzioni, il calcolo di un Upper Bound euristico è composto da una base logica che segue la procedura di Dijkstra per i cammini minimi unita con una logica di generazione di scenari alternativi. L'approccio è euristico in quanto l'algoritmo valuta solo un sottoinsieme di permutazioni, scelte casualmente, sia delle informazioni, sia dei target da raggiungere.

Le due principali innovazioni introdotte in questa versione consistono nella gestione di scenari multipli di esplorazione delle soluzioni e del meccanismo di stringimento dell'Upper Bound.

La prima novità riguarda la strutturazione di soluzioni seguendo scenari alternativi per la scelta del percorso. Qualora l'algoritmo incontri più strade per il raggiungimento di un nodo target di pari costo, non sceglie casualmente quale memorizzare e utilizzare ma, al contrario, crea uno scenario distinto per ogni strada ottima individuata. Con questa modalità, la ricerca del percorso ottimo per le informazioni e target successivi non risente di una decisione casuale, ma viene testata in parallelo insieme agli altri scenari possibili. Al termine del processo vengono estrapolati gli scenari migliori individuati e i relativi costi associati. Per rimanere in linea con l'ideologia di un algoritmo rapido ed euristico viene imposto un limite massimo sia alle soluzioni finali sia agli scenari da mantenere attivi durante l'esecuzione.

La seconda novità risiede nella convergenza dell'Upper Bound verso la soluzione ottima. La determinazione dell'Upper Bound migliore risulta particolarmente dispendiosa nella prima iterazione sebbene si utilizzi una procedura euristica. Per ovviare a questo problema si adotta una strategia di aggiornamento ripetuto in cui, a intervalli prefissati di iterazioni, l'Upper Bound viene ricalcolato con il medesimo codice, incorporando nei costi degli archi le penalità lagrangiane.

Algoritmo di Dijkstra con Controllo Capacità

Questa versione dell'algoritmo è stata ottimizzata utilizzando la libreria `heapq` di Python, la quale implementa una struttura dati con priorità per semplificare il codice.

```
tabella={}  
nodiDaVisitare=[]  
for nodo in N:  
    if nodo == sorgente:  
        tabella[sorgente]=[0,[]]  
    else:  
        tabella[nodo]=[math.inf,[]]  
heapq.heappush(nodiDaVisitare,(0,sorgente))  
nodiVisitati=[]  
while len(nodiDaVisitare)>0:  
    costo,nodo = heapq.heappop(nodiDaVisitare)  
    nodiVisitati.append(nodo)  
    archi = [(i, j) for i, j in A if i == nodo]  
    for i,j in archi:  
        if capacità[(i,j)]>=banda:  
            if tabella[j][0] > tabella[i][0]+costi[(i,j)]:  
                tabella[j][0] = tabella[i][0]+costi[(i,j)]  
                tabella[j][1]=[]  
                tabella[j][1].append(i)  
            if(j not in nodiVisitati):  
                heapq.heappush(nodiDaVisitare,(tabella[j][0],j))  
        else:  
            if tabella[j][0] == tabella[i][0]+costi[(i,j)]:  
                tabella[j][1].append(nodo)  
if tabella[target][0]==math.inf:  
    return []  
costo = tabella[target][0]  
percorsi = ricostruisciPercorsi(target,sorgente,tabella,k)  
return [(costo,percorso) for percorso in percorsi]
```

La chiamata alla funzione di Dijkstra modificata viene gestita da un metodo centrale che richiede in ingresso una permutazione delle informazioni. Per ciascuna di esse, la funzione itera su un numero prefissato di sottoinsiemi di target da raggiungere, mescolati casualmente, gestendo in contemporanea i diversi scenari attivi. Nello specifico, all'interno del metodo è stato inserito un meccanismo di controllo per garantire il passaggio gratuito attraverso un arco nel caso in cui questo fosse già stato attivato nello scenario corrente per la stessa informazione.

Per la creazione, si procede alla copia della capacità originale della rete e alla definizione della struttura che conterrà i diversi scenari.

```
capacità = u.copy()  
soluzioni = [[0,[],capacità,[],[]]]
```

La struttura è pianificata in modo da contenere per ogni scenario:

- Costo complessivo
- Archi attivi
- Capacità della rete residua
- Target raggiunti
- Target irraggiungibili

Successivamente, l'algoritmo itera su ogni informazione k e su ciascuna permutazione dei nodi target per la relativa informazione. Durante l'iterazione, il metodo considera tutti gli scenari precedentemente memorizzati e, per ogni target, cerca di espanderli. Se il cammino minimo verso il target è univoco lo scenario corrente viene aggiornato e si prosegue. Al contrario, se il percorso migliore non è unico, si crea un numero di scenari pari alle alternative ottime individuate. Qualora invece il target risulti non raggiungibile, il nodo viene catalogato come inarrivabile all'interno dello scenario.

```
for k in permutazione:
    soluzioniPerPermutazione = []
    for permutazioneTarget in permutazioniTarget:
        for soluzione in soluzioni:
            soluzioniTemp=[]
            soluzioniTemp.append(soluzione.copy())
            for target in permutazioneTarget:
                nuoviTemp=[]
                for soluzioneTemp in soluzioniTemp:
                    valori = copy.deepcopy(soluzioneTemp)
                    costiConsiderandoArchiAttivi = copy.deepcopy(C)
                    capacitàConsiderandoArchiAttivi = copy.deepcopy(soluzioneTemp[2])
                    archiAttivi = []
                    for kArco,i,j in soluzioneTemp[1]:
                        if k==kArco:
                            costiConsiderandoArchiAttivi[(i,j)]=0
                            capacitàConsiderandoArchiAttivi[(i,j)] += a[k]
                            archiAttivi.append((i,j))
                    percorsi = Dijkstra(NodoOrigine[k],target,
                    capacitàConsiderandoArchiAttivi,a[k],k,
                    costiConsiderandoArchiAttivi)
                    if len(percorsi)==0:
                        copia = copy.deepcopy(valori)
                        copia[4].append((k,target))
                        nuoviTemp.append
                        ([copia[0],copia[1],copia[2],copia[3],copia[4]])
                    else:
                        for i in range(len(percorsi)):
```

```

        costo , archi = percorsi[i]
        cap = nuovaCapacità(soluzioneTemp[2], archi, a[k]
        , archiAttivi)
        copia=copy.deepcopy(valori)
        copia[1].extend(archi)
        copia[3].append((k, target))
        nuoviTemp.append([copia[0]+costo, copia[1], cap, copia[3]
        , copia[4]])
        soluzioniTemp=nuoviTemp
        soluzioniPerPermutazione.extend(soluzioniTemp)
        soluzioniPerPermutazione.sort(key=lambda s: (-len(s[3]), s[0]))
        soluzioni=soluzioniPerPermutazione[:soluzioniDaContinuare]
return soluzioni

```

Per l'aggiornamento della capacità degli archi nella rete viene sviluppata una semplice funzione di riduzione dei valori.

```

cap = capacità.copy()
for arco in archi:
    if arco not in archiAttivi:
        arco = arco[1:3]
        cap[arco] -= banda

```

Per la ricostruzione del percorso si è optato per l'utilizzo di una funzione generatrice ricorsiva. Il metodo ripercorre la rete a ritroso restituendo, mediante l'istruzione yield, tutti i cammini minimi alternativi individuati.

```

if nodo == sorgente:
    yield []
    return
for predecessore in tabella[nodo][1]:
    for percorso in ricostruisciPercorsi(predecessore, sorgente, tabella, k):
        yield percorso + [(k, predecessore, nodo)]

```

Infine, la gestione delle permutazioni delle informazioni, la valutazione delle soluzioni e il corretto calcolo del costo della soluzione in caso di target non raggiunti sono coordinate da una funzione che funge come punto di partenza dell'intero algoritmo:

```

for permutazione in permutazioniInformazioni:
    soluzioni = MainEuristico(permutazione)
    soluzioniTotali.extend(soluzioni)
    if len(soluzioniTotali) > soluzioniMax:
        break
if len(soluzioniTotali)==0:
    return
maxRaggiunti = max(len(soluzione[3]) for soluzione in soluzioniTotali)
candidati = [soluzione for soluzione in soluzioniTotali if
len(soluzione[3])==maxRaggiunti]

```



```

costoMin = min(soluzione[0] for soluzione in candidati)
migliori = [soluzione for soluzione in candidati if soluzione[0]==costoMin]
costoMinModificato = costoMin+P*len(migliori[0][4])

```

Calcolo dei Costi con Penalità Lagrangiane

Per la convergenza dell'Upper Bound, il metodo centrale viene modificato per integrare nei pesi degli archi le penalità lagrangiane, calcolate dal metodo del rilassamento. È importante evidenziare che i costi utilizzati durante la fase di instradamento non sono più validi per il calcolo del costo complessivo della rete, poiché incrementati dal valore delle penalità λ . È quindi necessario ricalcolarli per determinare il costo della soluzione.

```

for soluzioneTemp in soluzioniTemp:
    costiLambda = {}
    for i,j in A:
        costiLambda[(i,j)] = C[(i,j)]+max(0,lam[(k,target,i)]-lam[(k,target,j)])
    for kArco, i, j in soluzioneTemp[1]:
        if k == kArco:
            costiLambda[(i,j)] = 0
        percorsi = Dijkstra(NodoOrigine[k],target,capacitàConsiderandoArchiAttivi,
a[k],k, costiLambda)
        if len(percorsi)==0:
            ...
        else:
            for i in range(len(percorsi)):
                costo,archi = percorsi[i]
                costo=0
                for kArco, nodoI, nodoJ in archi:
                    if (nodoI,nodoJ) not in archiAttivi:
                        costo += C[(nodoI,nodoJ)]
                nuoviTemp.append([copia[0]+costo,copia[1],cap,copia[3],copia[4]])

```

4.2.2 Terza formulazione

Lagrangiano

L'implementazione della terza formulazione usa come base le funzioni già sviluppate per la seconda, adattandole per consentire la gestione sia della variabile decisionale nuova sia di quelle vecchie modificate, in accordo con la teoria (3.2.2).

Calcola Y

A tale scopo, è stata definita una nuova funzione usata per determinare l'attivazione della variabile y , seguendo la condizione ricavata dal rilassamento lagrangiano (3.2.3).

```
for k in K:
    for t in NodiTarget[k]:
        if lam[(k, t, t)] - lam[(k, t, NodoOrigine[k])] < P:
            y[(k, t)] = 1
        else:
            y[(k, t)] = 0
```

Calcola B

Per rispettare la definizione teorica della variabile b (3.15), si implementa un metodo di aggiornamento che dipende dai valori assunti da y . Tale metodo viene richiamato a ogni iterazione, garantendo la coerenza tra le due variabili durante l'esecuzione dell'algoritmo.

```
b = {k: {t: {i: 0 for i in N} for t in NodiTarget[k]} for k in K}
for k in K:
    for t in NodiTarget[k]:
        b[k][t][NodoOrigine[k]] = y[(k, t)]
        b[k][t][t] = -y[(k, t)]
```

Calcolo Costi Penalizzati

La funzione per il calcolo dei costi penalizzati viene modificata in modo da restare conforme con l'aggiornamento della formula decisionale per l'attivazione degli archi (3.2.3). Quest'ultima, infatti, in aggiunta alla versione precedente, richiede un controllo di tutti i nodi target che si intendono raggiungere.

```
for (k, i, j) in d:
    costo = C[i, j]
    for t in NodiTarget[k]:
        if (lam[(k, t, i)] - lam[(k, t, j)]) < 0:
            costo += (lam[(k, t, i)] - lam[(k, t, j)])
    d[(k, i, j)] = costo
    if (d[(k, i, j)] < 0):
        ePerKnapsack.append((k, i, j))
```

Knapsack

All'interno del metodo implementato per la risoluzione del problema dello zaino, l'unico aggiornamento riguarda la determinazione del valore delle x che, in ca-

so di attivazione dell'arco, dovranno assumere un valore binario dipendente dalle penalità lagrangiane.

```

...
e[(k,i,j)] = 1.0
for t in NodiTarget[k]:
    if (lam[(k,t,i)]-lam[(k,t,j)]) < 0:
        x[(k,t,i,j)]=1
...

```

Calcolo del Lower Bound Attuale

L'algoritmo per il calcolo del Lower Bound viene riscritto per riflettere la nuova funzione obiettivo lagrangiana (3.22), derivata dal rilassamento della terza formulazione.

```

Lb=0.0
for (i,j) in A:
    for k in K:
        Lb += C[i,j]*e[(k,i,j)]
        for t in NodiTarget[k]:
            Lb += (lam[(k,t,i)]-lam[(k,t,j)])*x[(k,t,i,j)]
for k in K:
    for t in NodiTarget[k]:
        Lb += P*(1-y[(k,t)])
        for i in N:
            Lb -= lam[(k,t,i)]*b[k][t][i]

```

Calcolo dello Step e Aggiornamento delle Penalità

Per quanto riguarda il calcolo dello step e il successivo aggiornamento delle penalità lagrangiane, la logica dell'algoritmo rimane invariata. L'unica modifica introdotta, riguarda la dimensione delle variabili coinvolte che saranno indicizzate su tre parametri al posto dei precedenti due.

```

subGradienti = {(k,t,i):0.0 for k in K for t in NodiTarget[k] for i in N}
...
lam[(k,t,i)] = lam[(k,t,i)] + valoreDiCambio *subGradienti[(k,t,i)]

```

Main Loop

Il ciclo principale, invece, si modifica sia per gestire le chiamate alle varie funzioni descritte precedentemente sia per integrare la nuova variabile decisionale y all'interno della logica del problema.

```

y = {(k,t):0 for k in K for t in NodiTarget[k]}

```

```

lam = {(k,t,i):0.0 for k in K for t in NodiTarget[k] for i in N}
d = {(k,i,j):0 for k in K for (i,j) in A}
e = {(k,i,j):0 for k in K for (i,j) in A}
x = {(k,t,i,j):0.0 for k in K for t in NodiTarget[k] for (i,j) in A}
b = calcolaB(y)
ePerKnapsack = []
ultimiLowerBounds = [0]
Lb=0.0
alpha = 0.1
it_max=10000
alphaterazioni=0
iterazioni=0
UbEuristicoFormattato = TrovaSoluzioniEuristiche()
while abs(UbEuristicoFormattato-Lb)>1e-4 and iterazioni<it_max:
    iterazioni+=1
    ePerKnapsack=[]
    e = {(k,i,j):0 for k in K for (i,j) in A}
    x = {(k,t,i,j):0 for k in K for t in NodiTarget[k] for (i,j) in A}
    CalcolaY(lam, y)
    b = calcolaB(y)
    CalcolaCostiPenalizzati(lam,d,ePerKnapsack)
    KnapSack(ePerKnapsack,d,e,x,lam)
    Lb = calcolaMin(lam,b,y,e,x)
    ultimiLowerBounds.append(Lb)
    ...
    aggiornaLambda(lam,ultimiLowerBounds,UbEuristicoFormattato,alpha,x,b)

```

Cplex

Nel caso della terza formulazione, all'interno dell'ambiente Cplex, si è scelto di modificare la gestione delle variabili in modo da preservare la linearità e la leggibilità del codice. I due cambiamenti principali riguardano l'integrazione della variabile b all'interno del vincolo, mediante l'uso di condizioni, e la creazione di una struttura KT contenente le combinazioni di tutte le informazioni e i relativi target.

```

minimize
sum(k in K, <i,j> in A) C[<i,j>] * e[k][<i,j>] +
sum(k in K, t in NodiTarget[k]) P * (1 - y[k][t]);

subject to {

    forall(<k, t> in KT, i in N) {
        sum(j in Lt[i]) x[<k,t>][<i,j>] - sum(j in Lr[i]) x[<k,t>][<j,i>] ==
        (i == NodoOrigine[k] ? y[k][t] : (i == t ? -y[k][t] : 0));
    }

    forall(<i,j> in A)

```

```

sum(k in K) a[k] * e[k][<i,j>] <= u[<i,j>];

forall(<k, t> in KT, <i, j> in A) {
    x[<k,t>][<i,j>] <= e[k][<i,j>];
}
}

```

Python API

L'integrazione in linguaggio Python tramite l'API è stata sviluppata in modo da rispecchiare le modifiche effettuate precedentemente nell'ambiente Cplex. In primo luogo si creano le variabili decisionali e di supporto, garantendo nella definizione il rispetto dei loro vincoli associati (3.26) (3.27) (3.28).

```

modello = cplex.Cplex()
modello.objective.set_sense(modello.objective.sense.minimize)
KT = [(k, t) for k in K for t in NodiTarget[k]]
x = [(k,t,i,j) for (k, t) in KT for (i,j) in A]
modello.variables.add(names=[f"x_{k}_{t}_{i}_{j}" for (k,t,i,j) in x],
lb=[0]*len(x), types=["I"]*len(x))
e = [(k,i,j) for k in K for (i,j) in A]
modello.variables.add(names=[f"e_{k}_{i}_{j}" for (k,i,j) in e],
lb=[0]*len(e), ub=[1]*len(e), types=["B"]*len(e))
y = [(k,t) for (k, t) in KT]
modello.variables.add(names=[f"y_{k}_{t}" for (k,t) in y],
lb=[0]*len(y), ub=[1]*len(y), types=["B"]*len(y))

```

In secondo luogo, si definiscono la funzione obiettivo e i vincoli della formulazione, integrando la variabile b e sfruttando la struttura KT appena creata. Una particolarità nell'implementazione della funzione obiettivo riguarda l'introduzione di un offset da aggiungere al calcolo, ovvero di un valore costante indipendente dalle variabili decisionali. In questo caso specifico, si tratta del valore P moltiplicato per il numero di combinazioni di KT .

Funzione Obiettivo (3.22):

```

temp_X = []
temp_Coeffs=[]
for k in K:
    for (i,j) in A:
        temp_Coeffs.append(C[i,j])
        temp_X.append(f"e_{k}_{i}_{j}")
for (k, t) in KT:
    temp_Coeffs.append(-P)
    temp_X.append(f"y_{k}_{t}")
modello.objective.set_linear(list(zip(temp_X, temp_Coeffs)))
offset = P * len(KT)
modello.objective.set_offset(offset)

```

Primo vincolo (3.23)

```
for (k, t) in KT:
    for i in N:
        temp_X = []
        temp_Coeffs = []
        for j in Lt[i]:
            if (i, j) in A:
                temp_X.append(f"x_{k}-{t}-{i}-{j}")
                temp_Coeffs.append(1)
        for j in Lr[i]:
            if (j, i) in A:
                temp_X.append(f"x_{k}-{t}-{j}-{i}")
                temp_Coeffs.append(-1)
        if i == NodoOrigine[k]:
            temp_X.append(f"y_{k}-{t}")
            temp_Coeffs.append(-1)
        if i == t:
            temp_X.append(f"y_{k}-{t}")
            temp_Coeffs.append(1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["E"], rhs=[0])
```

Secondo vincolo (3.24):

```
for (i, j) in A:
    temp_X = []
    temp_Coeffs = []
    for k in K:
        temp_X.append(f"e_{k}-{i}-{j}")
        temp_Coeffs.append(a[k])
    modello.linear_constraints.add
    (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[u[i, j]])
```

Terzo vincolo (3.25):

```
for (k, t) in KT:
    for (i, j) in A:
        temp_X = []
        temp_Coeffs = []
        temp_X.append(f"x_{k}-{t}-{i}-{j}")
        temp_Coeffs.append(1)
        temp_X.append(f"e_{k}-{i}-{j}")
        temp_Coeffs.append(-1)
        modello.linear_constraints.add
        (lin_expr=[cplex.SparsePair(temp_X, temp_Coeffs)], senses=["L"], rhs=[0])
```

5 Analisi dei Risultati

In questo capitolo si approfondisce l'applicazione pratica degli algoritmi sviluppati, con l'obiettivo primario di valutarne l'efficacia e la correttezza all'interno di diversi scenari di test. Le istanze di test sono costruite per simulare una rete Publish/Subscribe [2], nella quale bisogna inviare simultaneamente più informazioni con la possibilità di target multipli e di nodi irraggiungibili.

Le soluzioni ottenute tramite gli algoritmi sviluppati vengono confrontate con il valore ottimo calcolato direttamente nell'ambiente Cplex [10] e la distanza da esso rappresenta una metrica fondamentale e sufficiente per valutarne la qualità.

La generazione degli scenari segue due modalità: un'arbitraria per analizzare il corretto funzionamento all'interno dei casi limite e una casuale per verificare la correttezza in contesti non preconfigurati. In particolare gli scenari casuali vengono generati con un incremento iterativo delle possibili variabili: informazioni, nodi, archi e numero target per individuare quale parametro porta criticità agli algoritmi.

5.1 Scenari e Test Arbitrari

Durante i vari test vengono raccolte statistiche secondarie, specificamente il tempo di esecuzione e il numero di iterazioni, per valutare, laddove possibile, l'andamento e l'efficienza degli algoritmi. Le implementazioni in Python basate sull'API di Cplex presentano, inoltre, un limite imposto dalla libreria, che consente l'esecuzione solo entro un prefissato numero di variabili.

5.1.1 Primo Scenario

La prima istanza di test è una rete semplice con un totale di 4 nodi. Le connessioni sono disposte in modo che i percorsi più brevi siano composti da soli due archi, tuttavia, i costi di questi collegamenti sono intenzionalmente elevati. L'obiettivo è vedere una scelta da parte degli algoritmi di un percorso più lungo al fine di ridurre i costi complessivi.

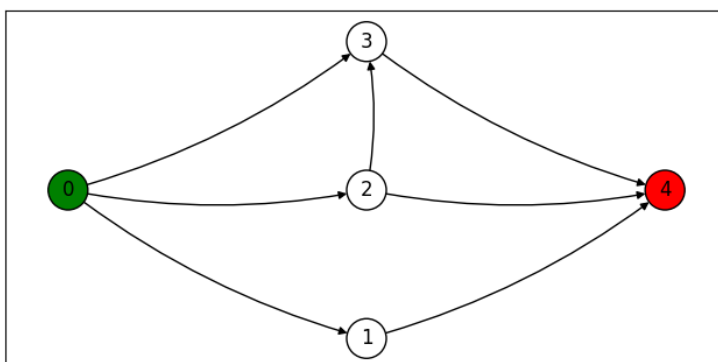


Figura 5.1: Primo scenario.

Dati Informativi

La rete deve trasportare una informazione 0 dal nodo Publisher 0 al Subscriber 4 potendo passare da 3 nodi Brokers. I costi del percorso $0 - 2 - 3 - 4$ sono molto ridotti.

Risoluzione

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Prima Formulazione Cplex	7	/	/
Prima Formulazione Python API	7	0.0115	/
Seconda Formulazione Lagrangiana	7	0.0173	208
Seconda Formulazione Cplex	7	/	/
Seconda Formulazione Python API	7	0.0580	/
Terza Formulazione Lagrangiana	7	0.0120	120
Terza Formulazione Cplex	7	/	/
Terza Formulazione Python API	7	0.0481	/
Upper Bound	7	/	/

Tabella 5.1: Panoramica dei risultati dei vari algoritmi per questo scenario.

Come emerge dai risultati, nonostante la prima formulazione e le successive due adottino criteri diversi per il calcolo del costo del percorso ottimo, hanno tutte restituito la medesima soluzione con tempi molto ridotti. Tale soluzione, inoltre, risulta ottima per ciascun algoritmo sviluppato, in quanto coincide con il risultato restituito dagli algoritmi sviluppati su Cplex. Le soluzioni tra le formulazioni coincidono in quanto, avendo una sola informazione e un solo target, la x si comporta similmente alla e e quindi le funzioni obiettivo coincidono.

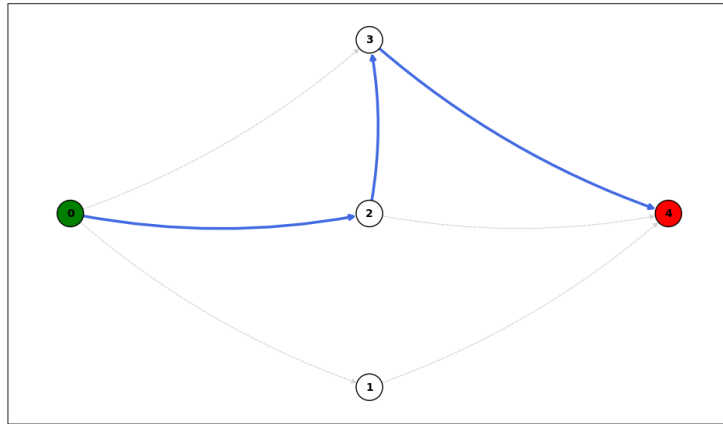


Figura 5.2: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.2 Secondo Scenario

Il secondo scenario di test viene utilizzato per analizzare il comportamento degli algoritmi in una rete densamente interconnessa. L'obiettivo è verificare la correttezza degli algoritmi nella scelta del percorso ottimo in un contesto con molteplici cammini alternativi.

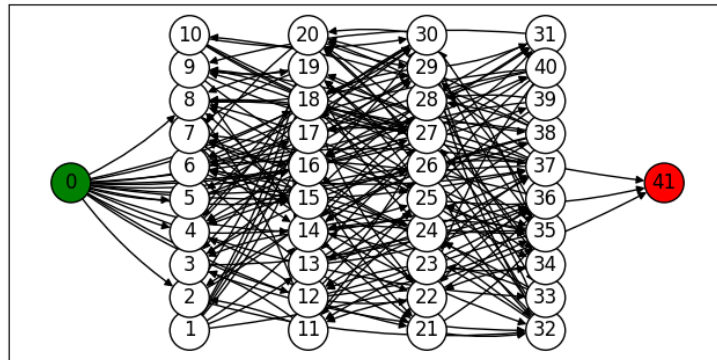


Figura 5.3: Secondo scenario.

Dati Informativi

La rete deve trasportare una informazione 0 dal nodo Publisher 0 al Subscriber 41 potendo passare da 40 nodi Brokers differenti utilizzando 201 possibili archi distinti. La peculiarità dello scenario risiede nell'elevato numero di archi all'interno del grafo, progettati per indurre una forte ridondanza e una molteplicità di cammini alternativi.

Risoluzione

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Prima Formulazione Cplex	113	/	/
Prima Formulazione Python API	113	0.0568	/
Seconda Formulazione Lagrangiana	113	0.9658	2700
Seconda Formulazione Cplex	113	/	/
Seconda Formulazione Python API	113	0.0500	/
Terza Formulazione Lagrangiana	113	0.8932	1640
Terza Formulazione Cplex	113	/	/
Terza Formulazione Python API	113	0.0549	/
Upper Bound	113	/	/

Tabella 5.2: Panoramica dei risultati dei vari algoritmi per questo scenario.

Come emerge dalle soluzioni, tutti gli algoritmi hanno trovato l'ottimo con tempi relativamente brevi, tuttavia, si rileva un peggioramento delle tempistiche rispetto al primo scenario, da associare sia all'aumento dei percorsi da controllare tramite l'algoritmo di Upper Bound, sia all'aumento delle iterazioni dei cicli, i quali dipendono dal numero di archi e nodi. Analogamente a quanto osservato nel primo scenario, l'uguaglianza tra le soluzioni delle diverse formulazioni dipende dal comportamento della variabile x in scenari con una sola informazione da trasmettere a un solo target.

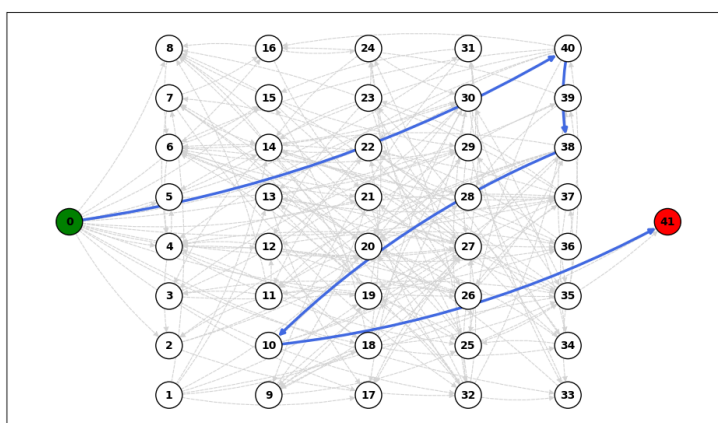


Figura 5.4: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.3 Terzo Scenario

Il terzo scenario di test introduce una situazione di congestione della rete in cui tre informazioni, di cui una con due target, competono per arrivare ai nodi Subscriber. Le capacità degli archi sono state elaborate precisamente per indurre uno scenario di saturazione che colpisca il percorso ottimo, costringendo agli algoritmi di dover usufruire di altri cammini per la trasmissione delle informazioni. Gli obiettivi di questo scenario consistono nel verificare la corretta saturazione della cammino ottimo e l'efficace scelta della via percorribile alternativa.

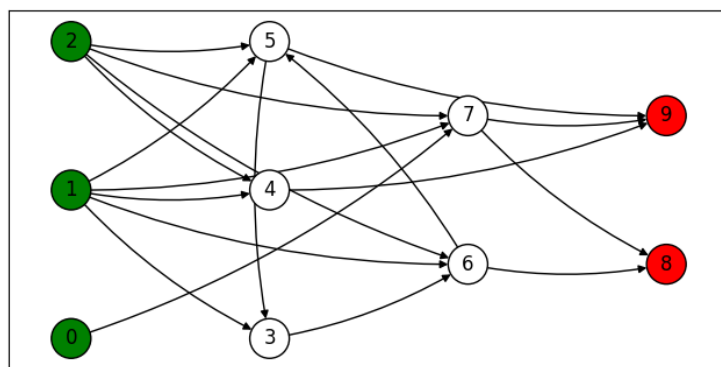


Figura 5.5: Terzo scenario

Dati Informativi

La rete deve trasportare tre informazioni 0, 1, 2 ai rispettivi nodi target. Nello specifico l'informazione 0 parte dal Publisher 1 e deve raggiungere i target 8 e 9, l'informazione 1 parte dal Publisher 2 e deve raggiungere il target 9 e, infine, l'informazione 2 parte dal Publisher 0 e deve raggiungere il target 9. Il percorso ottimo per il raggiungimento del nodo 9 prevede il transito attraverso il nodo 7, tuttavia, la sua capacità è stata intenzionalmente limitata per simulare una saturazione.

Risoluzione

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Prima Formulazione Cplex	229	/	/
Prima Formulazione Python API	229	0.0799	/
Seconda Formulazione Lagrangiana	233.73	2.0903	10000
Seconda Formulazione Cplex	234	/	/
Seconda Formulazione Python API	234	0.0536	/
Terza Formulazione Lagrangiana	234	0.2831	942
Terza Formulazione Cplex	234	/	/
Terza Formulazione Python API	234	0.0818	/
Upper Bound	234	/	/

Tabella 5.3: Panoramica dei risultati dei vari algoritmi per questo scenario.

Dall'analisi delle soluzioni restituite dagli algoritmi emerge una mancata convergenza dell'algoritmo lagrangiano della seconda formulazione entro il limite massimo di iterazioni prefissate. Andando nel dettaglio, si nota che, mentre l'Upper Bound restituisce il risultato ottimo, il valore della soluzione lagrangiana si ferma a 233.73. Al contrario, la terza formulazione mostra un'eccellente efficienza computazionale, convergendo all'ottimo in tempi e iterazioni ridotte. Controllando la soluzione ottima, si riscontra il comportamento ricercato dallo scenario, ovvero la saturazione del cammino dal nodo 7 a 9, che costringe l'informazione in partenza dal nodo 2 a passare dal nodo 4 per raggiungere il nodo 9, nonostante il tragitto sia più costoso. Il motivo del blocco a una determinata soglia da parte del Lower Bound della seconda formulazione dipende dalla difficoltà nelle casistiche in cui una informazione richiede la trasmissione a più target. Questo avviene a causa della formulazione debole dei vincoli, infatti, eliminando la doppia richiesta l'algoritmo converge senza problemi.

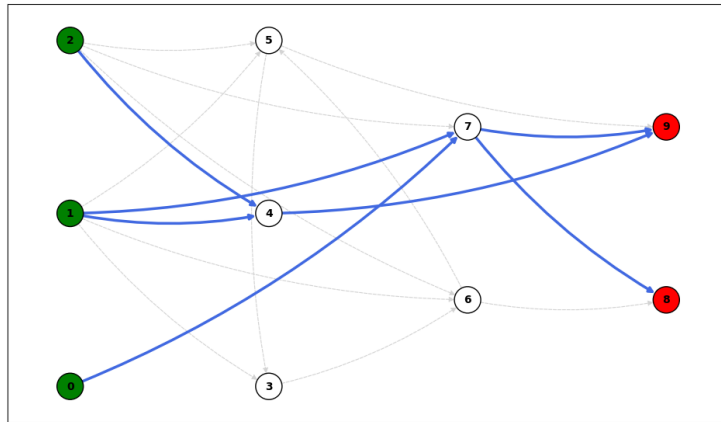


Figura 5.6: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.4 Quarto Scenario

Il quarto scenario di test rappresenta una rete caratterizzata dalla saturazione dei percorsi ottimi per il raggiungimento di un nodo target. L'obiettivo dello scenario è di valutare la capacità dell'algoritmo derivante dalla terza formulazione di selezionare opportunamente quali informazioni instradare fino a destinazione e quali invece escludere, verificando parallelamente l'efficacia dell'algoritmo nella scelta dei percorsi ottimi.

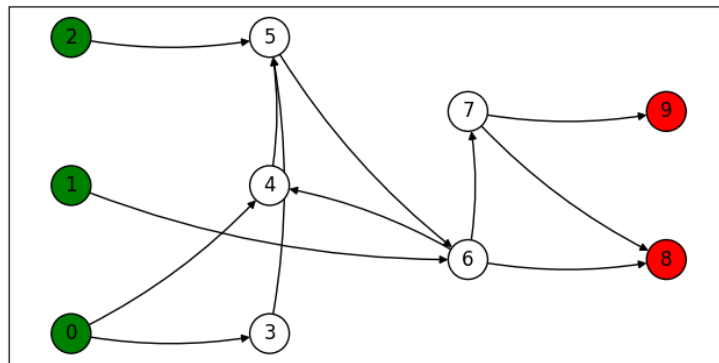


Figura 5.7: Quarto scenario

Dati Informativi

La rete deve trasportare tre informazioni 0, 1, 2 ai rispettivi nodi target. Nello specifico l'informazione 0 parte dal Publisher 1 e deve raggiungere i target 8 e 9, l'informazione 1 parte dal Publisher 0 e deve raggiungere il target 8, e l'informazione 2 parte dal Publisher 2 e deve raggiungere i target 8 e 9. I percorsi ottimi per il raggiungimento dei nodi 9 e 8 hanno la capacità intenzionalmente limitata per impedire il corretto arrivo di tutte le informazioni ai relativi target. La scelta che si vuole indurre è il corretto reinstradamento delle informazioni destinate al target 8 e il mancato invio di una informazione per il raggiungimento del target 9.

Risoluzione

In quanto lo scenario si basa sulla mancata ricezione da parte di un'informazione l'unico algoritmo che fornisce soluzione per il Lower Bound è quello basato sulla terza formulazione. È importante sottolineare che, anche qualora non fosse stata necessaria una soluzione parziale, la seconda formulazione si sarebbe dimostrata comunque inadeguata per via della richiesta da parte di molteplici target della medesima informazione.

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Terza Formulazione Lagrangiana	1026	0.4830	1331
Terza Formulazione Cplex	1026	/	/
Terza Formulazione Python API	1026	0.0732	/
Upper Bound	1026	/	/

Tabella 5.4: Panoramica dei risultati dei vari algoritmi per questo scenario.

Come evince dai risultati, l'algoritmo lagrangiano ha individuato la soluzione ottima in tempi ridotti. Nello scenario corrente, la scelta ottima ricade nel non consegnare l'informazione 2 al target 9. La decisione presa è dettata dalla topologia della rete, la quale richiederebbe all'informazione 2 di attraversare un tragitto più lungo e costoso rispetto all'informazione 0. Inoltre i costi risultano molto elevati perché sono sommati alla penalità associata col mancato trasferimento di una informazione che nel quarto scenario vale 553.

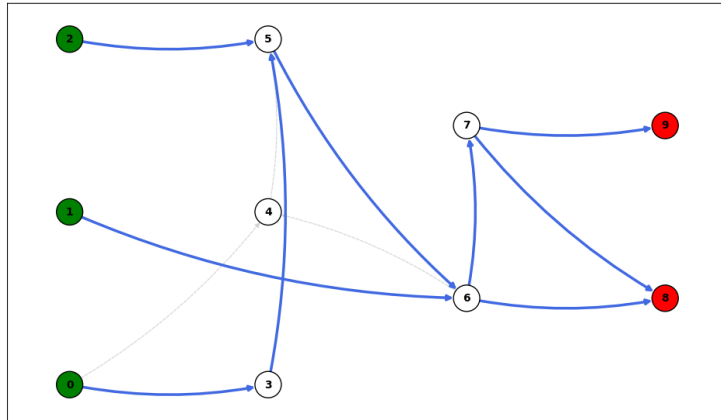


Figura 5.8: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.1.5 Quinto Scenario

Il quinto scenario rappresenta una rete caratterizzata da un solo arco collegato con un target richiesto da tutte le informazioni transittanti. L'obiettivo dello scenario è di mostrare una preferenza, sia da parte dell'algoritmo lagrangiano sia da parte di quello euristico, a favore dell'invio di un numero maggiore di informazioni con una minore occupazione di banda, piuttosto che la trasmissione di una singola informazione che saturerebbe l'arco.

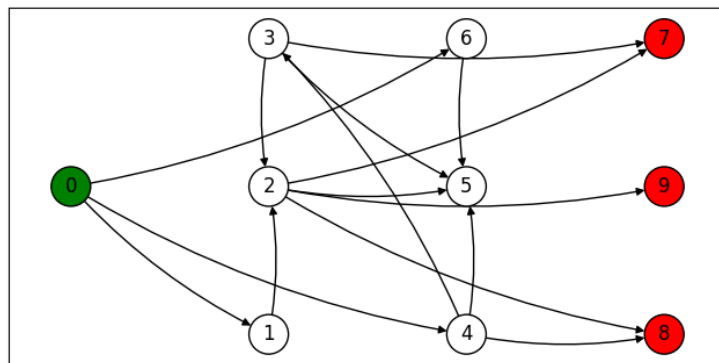


Figura 5.9: Quinto scenario

Dati Informativi

La rete deve trasportare tre informazioni 0, 1, 2 ai rispettivi target. Nello specifico, tutte e tre vengono richieste dal nodo target 9, il quale è collegato con il resto della

rete da un singolo arco di capacità intenzionalmente limitata. Il collegamento può sostenere una banda massima pari a 4 e le informazioni occupano rispettivamente 2, 2 e 4. Quello che si vuole mostrare è che gli algoritmi preferiscono instradare le due informazioni con minore occupazione rispetto all'invio della singola che saturerebbe l'arco, favorendo la soddisfazione di più richieste.

Risoluzione

In quanto lo scenario richiede una soluzione parziale, l'unico algoritmo che restituisce un risultato per il Lower Bound è quello derivante dalla terza formulazione.

Algoritmo	Soluzione	Tempo (secondi)	Iterazioni
Terza Formulazione Lagrangiana	1458	4.6466	10000
Terza Formulazione Cplex	1458	/	/
Terza Formulazione Python API	1458	0.0582	/
Upper Bound	1465	/	/

Tabella 5.5: Panoramica dei risultati dei vari algoritmi per questo scenario.

I risultati mostrano una convergenza parziale verso la soluzione ottima, nello specifico, mentre il Lower Bound converge completamente, l'Upper Bound si attesta su un valore pari a 1465. Nonostante la distanza dall'ottimo, l'esito dello scenario è da considerare eccellente. La distanza dall'ottimo è di sole 7 unità, valore molto minore rispetto alla mediana del costo degli archi che vale 57, dimostrando che, anche in caso di mancata convergenza completa, il risultato ottenuto è comunque migliore di una soluzione che commetterebbe un errore pari al costo di un singolo arco per l'instradamento. Il successo dello scenario inoltre si può confermare sia dalla piena convergenza del Lower Bound sia dall'analisi del percorso ottimo restituito dall'Upper Bound, in cui l'unica informazione non trasmessa è la 2 per il target 9.

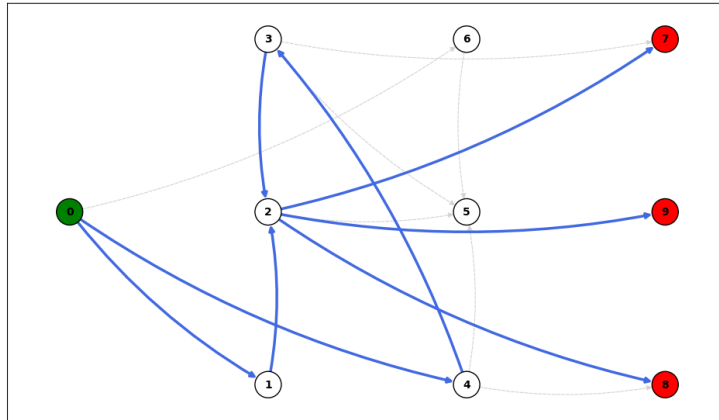


Figura 5.10: Percorso ottimo disegnato sulla base del risultato della terza formulazione.

5.2 Scenari e Test Casuali

Come descritto in precedenza, questa sezione è dedicata allo sviluppo di scenari casuali con la peculiarità di un incremento progressivo, a ogni iterazione, di una specifica variabile della rete. A causa dell'elevata probabilità di nodi irraggiungibili e target multipli in quanto generati casualmente, si è optato per applicare il test solo sulla terza formulazione ovvero quella centrale di questa tesi, oltre che quella che ha dato i migliori risultati negli scenari arbitrari. Questo approccio permette di testare gli algoritmi in simulazioni sempre più complesse, portando alla luce eventuali limiti o difficoltà.

5.2.1 Scenario Incrementale Casuale

Per valutare l'ottimalità delle soluzioni anche nei casi di mancata convergenza, all'interno della tabella è stato introdotto il parametro mA che rappresenta il costo mediano degli archi in quella precisa istanza. L'aggiunta di tale metrica permette di stabilire una soglia di tolleranza quale, se la distanza tra Lower e Upper risulta inferiore al costo mediano degli archi, allora si può considerare trascurabile in quanto vale meno di un errore sulla scelta di un singolo arco.

Numero Iterazione	Incremento Effettuato	Risultato Lower Bound	Breve Descrizione
1	Target	632	Lower, Upper ottimi
2	Target	796	Lower, Upper ottimi
3	Target	3596	Lower, Upper ottimi
4	Informazioni	6396	Lower ottimo,gap=36,mA=40
5	Broker	6396	Lower, Upper ottimi
6	Archi	6396	Lower, Upper ottimi
7	Informazioni	9196	Lower, Upper ottimi
8	Broker	9196	Lower ottimo,gap=36,mA=40.5
9	Archi	9196	Lower ottimo,gap=96,mA=41
10	Broker	9186.66	Fallito

Tabella 5.6: Tabella dei risultati a ogni istanza degli scenari casuali.

I risultati raccolti nella tabella permettono di valutare vari aspetti del nostro algoritmo lagrangiano per la terza formulazione all'aumentare della complessità dello scenario. Nelle prime iterazioni, l'incremento dei target aumenta vertiginosamente il costo della rete, evidenziando come alcuni nodi aggiunti sono irraggiungibili. Un comportamento simile si nota con l'aggiunta di nuove informazioni da trasmettere, ciò è dovuto sia agli archi attivati per soddisfare le nuove richieste sia all'eventuale impossibilità di raggiungimento dei target. Al contrario di questi eventi, l'aumento della dimensione della rete non provoca alterazioni nel valore ottimo. Questo accade perché, a causa della generazione casuale, è altamente improbabile che un nuovo nodo o arco riescano a creare un percorso ottimo. Analizzando più nel dettaglio la convergenza, si nota come, nelle casistiche in cui l'Upper Bound non è riuscito a raggiungere l'ottimo, il Lower Bound abbia comunque dimostrato una convergenza più stabile. In due di questi casi, la distanza tra il Lower ottimo e l'Upper è meno costosa di un errore di scelta su un arco, rendendo quindi ottimale lo scenario. Solo alla fine, con l'aggiunta di un Broker, si è notevolmente complicato il calcolo dell'Upper Bound, allontanandolo dall'ottimo. Questa distanza ha ridotto l'efficacia dello step di aggiornamento del Lower Bound, impedendone la corretta convergenza all'interno del numero di iterazioni prefissate. Da questa analisi emerge la necessità di approfondire separatamente due aspetti: l'aumento di informazioni e di target e l'aumento di nodi e di archi.

5.2.2 Scenario Incrementale Informazioni e Target

Numero Iterazione	Incremento Effettuato	Risultato Lower Bound	Breve Descrizione
1	Informazioni	1073	Lower, Upper ottimi
2	Informazioni	2045	Lower, Upper ottimi
3	Target	5156	Lower, Upper ottimi
4	Informazioni	6167	Lower, Upper ottimi
5	Informazioni	7178	Lower, Upper ottimi
6	Target	12257	Lower, Upper ottimi
7	Informazioni	13270	Lower, Upper ottimi
8	Informazioni	14283	Lower, Upper ottimi
9	Target	20802	Lower, Upper ottimi

Tabella 5.7: Tabella dei risultati a ogni istanza degli scenari casuali.

Dai risultati raccolti si può evidenziare un comportamento prevedibile da parte dei nostri algoritmi, sia all'aumentare delle informazioni sia dei target. Questo è dovuto al fatto che, avendo una struttura semplice della rete, individuare i percorsi ottimi risulta facile. Una volta individuati, tutte le informazioni che non riescono a raggiungere i target vengono semplicemente tramutate nelle penalità da aggiungere al costo totale.

5.2.3 Scenario Incrementale Broker e Archi

Numero Iterazione	Incremento Effettuato	Risultato Lower Bound	Breve Descrizione
1	Archi	110	Lower, Upper ottimi
2	Archi	110	Lower, Upper ottimi
3	Archi	110	Lower, Upper ottimi
4	Broker	110	Lower, Upper ottimi
5	Archi	110	Lower, Upper ottimi
6	Broker	110	Lower, Upper ottimi

Tabella 5.8: Tabella dei risultati a ogni istanza degli scenari casuali.

Dai risultati nella tabella risulta chiaro come la condizione analizzata precedentemente valga allo stesso modo anche per l'aumento dei nodi e degli archi. Infatti, nonostante la rete diventi più complessa, l'informazione da inviare e il nodo da raggiungere rimangono fissi. Poiché l'aggiunta di nodi o archi può solo mantenere o migliorare la soluzione esistente, il problema rimane facilmente risolvibile.

5.3 Analisi della Sensibilità dei Parametri

In questa sezione si analizza l'impatto dei principali parametri configurabili sulle prestazioni degli algoritmi lagrangiani sviluppati. In particolare, i parametri presi in esame sono α per l'aggiornamento delle penalità lagrangiane, la qualità dell'Upper Bound fornito alla formula per il calcolo dello step e le metriche per definire la profondità della ricerca euristica. Per misurare correttamente l'andamento della convergenza, è stata presa in esame una rete generata casualmente, risolta utilizzando l'algoritmo lagrangiano basato sulla terza formulazione.

5.3.1 Alpha

Il parametro α controlla l'ampiezza dello step applicato per l'aggiornamento delle penalità lagrangiane secondo la formula (2.28). Un valore elevato di α produce aggiornamenti più ampi quindi che accelerano la convergenza nella fase iniziale, ma possono causare oscillazioni attorno all'ottimo impedendo una convergenza precisa. Al contrario, un valore ridotto garantisce una convergenza più stabile ma più lenta, richiedendo un numero maggiore di iterazioni.

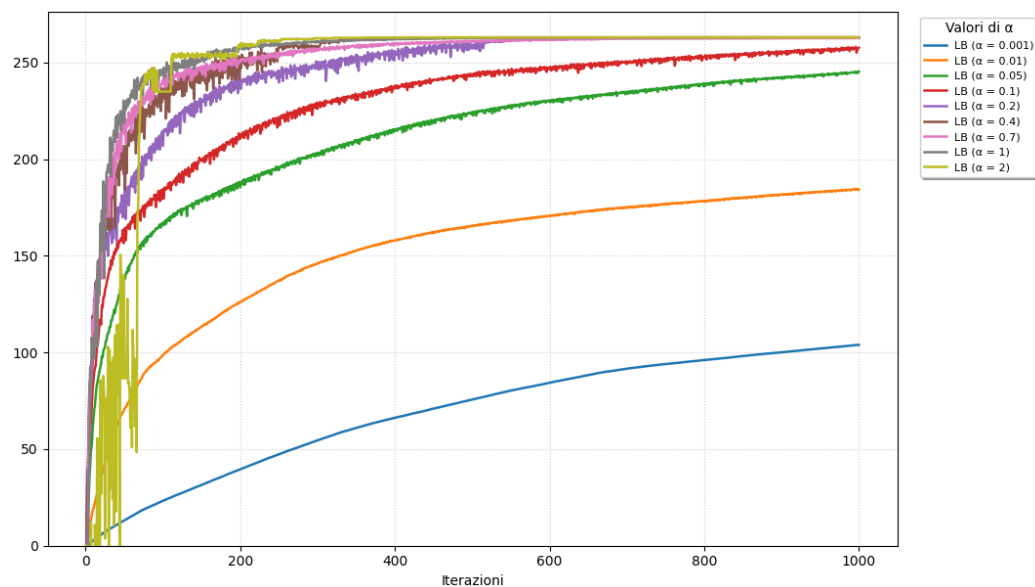


Figura 5.11: Convergenza del Lower Bound al variare di α .

Dal grafico emergono tre macro-comportamenti distinti in funzione del valore di α scelto:

- Se α ha un valore iniziale molto ridotto, la curva prodotta risulta regolare e priva di oscillazione, ma con una velocità di convergenza molto bassa.

- Se α ha un valore iniziale molto elevato, la curva prodotta cresce in maniera vertiginosa, raggiungendo velocemente i valori prossimi all'ottimo ma producendo forti oscillazioni. Normalmente queste oscillazioni impedirebbero una stabilizzazione precisa del Lower Bound ma il problema viene risolto con la funzione di dimezzamento (4.1.3).
- Se α ha un valore iniziale nella fascia intermedia, la curva prodotta risulta un buon compromesso tra le precedenti due, presentando una buona velocità di convergenza e oscillazioni contenute ma presenti.

5.3.2 Qualità dell'Upper Bound

La qualità dell'Upper Bound fornito all'algoritmo lagrangiano influenza direttamente l'ampiezza dello step di aggiornamento delle penalità. Poiché lo step è proporzionale alla differenza tra Upper Bound e Lower Bound seguendo la formula (2.28), un Upper Bound sovrastimato rischia di produrre step eccessivamente ampi, soprattutto nella fase finale della convergenza quando il Lower Bound si è già avvicinato all'ottimo.

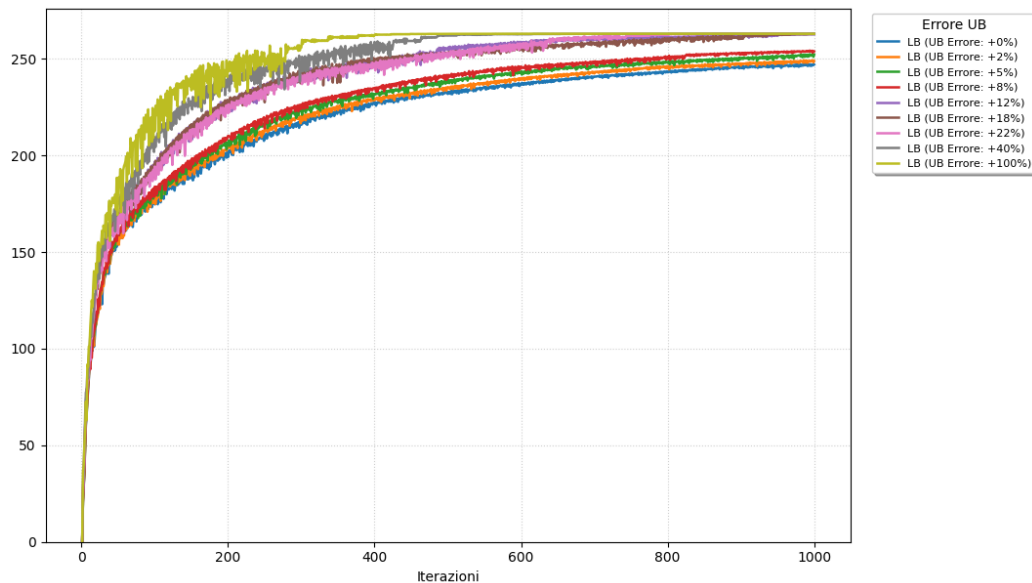


Figura 5.12: Convergenza del Lower Bound al variare dell'Upper Bound

Dal grafico emerge come l'effetto prodotto da una sovrastima dell'Upper Bound sia analogo a quello di un aumento del parametro α . Più l'Upper Bound risulta distante dall'ottimo, più la curva di convergenza sarà vertiginosa e avrà oscillazioni più estreme. Al contrario, con un Upper Bound ottimale si ha una curva di convergenza

più tranquilla e con oscillazioni più contenute. Questo test conferma inoltre come, nonostante un calcolo impreciso dell'Upper Bound, l'algoritmo lagrangiano sia comunque in grado di trovare il Lower Bound ottimale, almeno su reti di semplice complessità.

5.3.3 Profondità della Ricerca Euristica

L'algoritmo euristico per il calcolo dell'Upper Bound è gestito da due parametri principali che ne controllano la profondità di ricerca:

- **Massime Permutazioni:** Il numero massimo di permutazioni dei target considerate per ciascuna informazione.
- **Massime Soluzioni:** Il numero massimo di scenari alternativi mantenuti attivi durante l'esecuzione dell'algoritmo.

In linea teorica, aumentare questi parametri migliora la qualità dell'Upper Bound trovato, a costo di un incremento del tempo di ricerca.

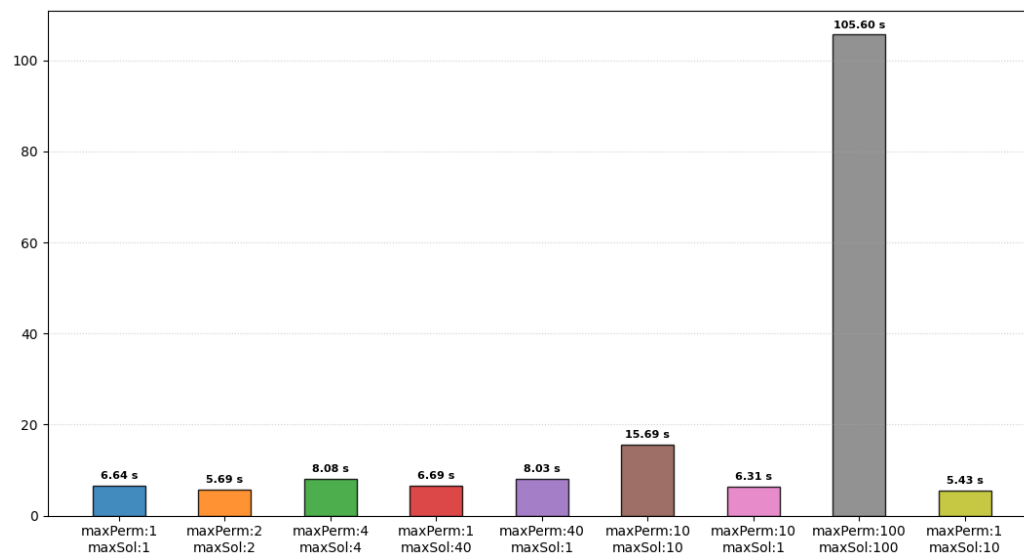


Figura 5.13: Grafico mostrante i tempi di esecuzione dell'algoritmo euristico in base ai valori dei parametri.

Dal grafico emerge come i tempi di esecuzione crescano all'aumentare dei parametri, ma con un comportamento asimmetrico tra i due. Incrementare esclusivamente le massime permutazioni produce tempi leggermente superiori rispetto all'incremento esclusivo delle massime soluzioni. Tuttavia, l'aumento singolo di

ciascuna delle due variabili, non produce un incremento significativo se confrontato con quello derivante dall'aumento contemporaneo di entrambe. Dal grafico questo fenomeno risulta evidente: configurazioni con un solo parametro elevato, anche di molto, producono tempi comunque inferiori rispetto a configurazioni in cui entrambi i parametri assumono valori anche ridotti. Questo conferma una crescita strettamente dipendente dal prodotto dei due parametri, infatti, per ogni permutazione considerata vengono esplorati tutti gli scenari attivi.

5.3.4 Conclusioni sui Test

I risultati ottenuti nei diversi scenari testati consentono di trarre delle considerazioni conclusive sull'efficacia e sui limiti degli algoritmi sviluppati.

Negli scenari arbitrari, la terza formulazione si è dimostrata la migliore, non solo avendo la possibilità di gestire più tipologie di scenari rispetto alle altre due, ma convergendo anche in tempi minori garantendo comunque il corretto instradamento delle informazioni.

Negli scenari casuali incrementali emerge un risultato interessante e coerente con la teoria, l'aumento singolo di informazioni e target o di nodi e archi non induce una complessità sufficiente a mettere in difficoltà gli algoritmi. La criticità emerge quando più variabili vengono aumentate in contemporanea, richiedendo sia la soddisfazione di più richieste sia una elaborazione delle stesse in una rete più articolata.

L'analisi dei parametri, invece, ha fornito indicazioni pratiche utili per la calibrazione dello step e dell'algoritmo euristico.

Il parametro α incide significativamente sulla velocità e sulla stabilità della convergenza, così come la distanza dell'Upper Bound dall'ottimo. Scegliere un α troppo elevato rischia di complicare la convergenza per colpa delle oscillazioni. Al contrario, una scelta più ridotta di α porta a una convergenza più stabile ma lenta.

Un effetto parzialmente analogo si osserva con la distanza dell'Upper Bound dall'ottimo, cioè una sovrastima del valore produce oscillazioni più estreme e rischia di rallentare la convergenza del Lower Bound.

Per quanto riguarda la profondità della ricerca euristica, i test confermano che il costo computazionale è gestito da due parametri fondamentali: il numero di permutazioni da controllare per informazione e il numero di scenari attivi da mantenere. Incrementare un solo parametro alla volta non produce un aumento significativo dei tempi di calcolo, mentre, al contrario, l'aumento di entrambi genera una crescita dei tempi molto più marcata.

In conclusione, i test evidenziano come la terza formulazione e gli algoritmi sviluppati siano efficaci in numerosi scenari e riescano a ottenere i risultati attesi.

Negli scenari in cui l'Upper Bound non converge all'ottimo, il Lower Bound riesce quasi sempre, dimostrando come la criticità principale risieda nel calcolo di un Upper Bound euristico di alta qualità. Poiché l'Upper Bound viene ricalcolato periodicamente sfruttando le penalità lagrangiane, si tende a ottenere la migliore soluzione individuabile da un approccio basato sui cammini minimi, che non sempre corrisponde all'ottimo. Nonostante questa limitazione, gli algoritmi rimangono utilizzabili in modo efficiente in contesti realistici, producendo soluzioni di buona qualità in tempi contenuti.

6 Conclusioni e Sviluppi Futuri

Questa tesi evidenzia come la trasmissione di informazioni all'interno di una rete, in particolar modo negli ambienti IoT [1], caratterizzati da un'elevata vulnerabilità a guasti o richieste multiple, possa trovarsi davanti a scenari critici. In tali contesti, nasce l'esigenza di definire un approccio ottimizzato in grado di determinare i percorsi corretti di instradamento delle informazioni gestendo la possibilità di richieste irrisolvibili attraverso la generazione di soluzioni parziali.

La tesi propone un approccio basato sulla convergenza di un Lower Bound e un Upper Bound per individuare la soluzione ottima. Il Lower Bound viene calcolato mediante un algoritmo sviluppato a partire dal rilassamento lagrangiano [5] della formulazione matematica del problema (3.2.4). L'Upper Bound, invece, deriva da un algoritmo euristico basato sulla logica di Dijkstra [7] per l'individuazione dei cammini minimi, esteso alla gestione di scenari alternativi.

I test effettuati hanno dimostrato una convergenza ottima e robusta della metodologia proposta, grazie soprattutto all'efficacia dell'algoritmo lagrangiano. Al contrario, l'aumento della complessità della rete tende a penalizzare la fase di calcolo dell'Upper Bound, confermandolo come l'anello debole nella ricerca della soluzione ottima. Nonostante ciò, l'approccio si rileva efficiente, garantendo soluzioni ottime o di buona qualità in tempi computazionalmente ridotti, anche su reti di media e alta complessità.

Possibili sviluppi futuri di questo lavoro includono una rielaborazione della logica alla base dell'algoritmo di Upper Bound, al fine di garantire una sua corretta convergenza anche in casi di elevata complessità. Parallelamente, si potrebbe procedere allo sviluppo distribuito dell'algoritmo lagrangiano, spostando il calcolo da un server centralizzato ai singoli nodi della rete.

In conclusione, la soluzione presentata in questa tesi si dimostra un approccio robusto ed efficiente per l'instradamento in reti Publish/Subscribe [2] ponendo basi solide per un suo futuro impiego in contesti reali.

Bibliografia

- [1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer Networks*, 54(15):2787–2805, 2010.
- [2] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The many faces of publish/subscribe. *ACM Comput. Surv.*, 35(2):114–131, June 2003.
- [3] MQTT. <https://mqtt.org/>.
- [4] Leonardo Urbinati. *Study of distributed Lagrangian heuristics for self-adaptive publish/subscribe network design problems*. PhD thesis.
- [5] Marshall L. Fisher. The lagrangian relaxation method for solving integer programming problems. *Management Science*, 27(1):1–18, 1981.
- [6] Laurence A. Wolsey. *Integer Programming*. Wiley, 1998.
- [7] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [8] Pietro Manzoni, Vittorio Maniezzo, and Marco A. Boschetti. Modeling distributed mqtt systems using multicommodity flow analysis. *Electronics*, 11(9), 2022.
- [9] Rüdiger Schollmeier. A definition of peer-to-peer networking for the classification of peer-to-peer architectures and applications. In *Proceedings First International Conference on Peer-to-Peer Computing*, pages 101–102. IEEE, 2001.
- [10] Cplex Optimization Studio IBM. <https://www.ibm.com/products/ilog-cplex-optimization-studio>.
- [11] Google. Google colaboratory. <https://colab.research.google.com>.
- [12] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. NetworkX: Network analysis in Python, 2008. <https://networkx.org>.
- [13] Matplotlib. <https://matplotlib.org/>.
- [14] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [15] Silvano Martello and Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.